Bhai, **"Longest Substring Without Repeating Characters"** interviews ka ek behad pasandida sawal hai. Iska brute force tarika samajhna isliye zaroori hai taaki jab tum iske baad "Sliding Window" (optimized) approach par jao, toh tumhe poora logic crystal clear ho jaye.

Chalo iske brute force tarike ko poori detail mein, intuition aur dry run ke saath samajhte hain!

🎯 Problem Kya Hai?

Aapko ek string di jayegi, aur aapko sabse lambi aisi **substring** (lagatar akshar) dhoondni hai jisme koi bhi character **dobara (repeat)** na aaye.

- **Input:** `s = "pwwkew"`
- **Output:** `3` (Kyunki sabse lambi bina repetition waali substring `"wke"` hai, jiski length 3 hai).

🧠 Brute Force Intuition: "Check Every Possibility"

Brute force ka seedha funda hota hai—kuch mat sochao, saari possibilities ko ek-ek karke check karo.

Hum string ke **har ek character ko starting point** banayenge aur wahan se aage badhte hue sabse lambi valid substring dhoondne ki koshish karenge.

Steps Kya Honge?

1. Ek variable banao `max_len = 0` jo hamari sabse lambi substring ki length ko track karega.
2. **Outer Loop (`i`):** Yeh tay karega ki hamari substring kahan se **shuru** ho rahi hai (index `0` se lekar `n-1` tak).
3. **Inner Loop (`j`):** Yeh `i` se shuru hoga aur aage badhega (substring ko **expand** karega).
4. Har step par, hum ek frequency array ya set rakhenge jo batayega ki jo character `s[j]` par hai, kya woh hum pehle dekh chuke hain?

- **Agar naya character hai:** Toh use apne set/array mein mark karo aur ab tak ki length (`j - i + 1`) ko `max_len` ke saath update kar lo.

- **Agar repeat character mila:** Iska matlab ab yeh substring valid nahi rahi. Inner loop ko yahin **break** kar do aur agle starting point (`i+1`) par jao.

🔍 Detailed Dry Run (Example: `s = "pwwkew"`)

Initial State: `max_len = 0`

- **Case 1: `i = 0` (Starting with 'p')**
- `j = 0` (`s[j] = 'p'`): Naya hai. `max_len = max(0, 0-0+1) = 1`.
- `j = 1` (`s[j] = 'w'`): Naya hai. `max_len = max(1, 1-0+1) = 2`.
- `j = 2` (`s[j] = 'w'`): **Repeat mila!** Kyunki 'w' hum pehle dekh chuke hain. Inner loop break.
- **Case 2: `i = 1` (Starting with 'w')**
- `j = 1` (`s[j] = 'w'`): Naya hai (is naye loop ke liye). `max_len = max(2, 1-1+1) = 2`.
- `j = 2` (`s[j] = 'w'`): **Repeat mila!** Loop break.
- **Case 3: `i = 2` (Starting with 'w')**
- `j = 2` (`s[j] = 'w'`): Naya hai. `max_len = max(2, 2-2+1) = 2`.
- `j = 3` (`s[j] = 'k'`): Naya hai. `max_len = max(2, 3-2+1) = 2`.
- `j = 4` (`s[j] = 'e'`): Naya hai. `max_len = max(2, 4-2+1) = 3`. (Substring bani `"wke"`)
- `j = 5` (`s[j] = 'w'`): **Repeat mila!** Loop break.

Aise hi baaki indices ke liye bhi check hoga, par `max_len` 3 se bada nahi ho payega. Final Answer = **3**.

💻 C++ Code (Brute Force)

Yahan iska simple aur clear C++ implementation hai, jahan humne character track karne ke liye ek boolean array (size 256) use kiya hai:

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstringBrute(string s) {
    int n = s.length();
    int max_len = 0;

    // Outer Loop: Har ek index ko starting point maano
    for (int i = 0; i < n; i++) {

        // Ek fixed size ka array jo check karega ki koi char repeat toh
nahi hua
        // Har naye starting point ke liye naya array banega
        bool visited[256] = {false};

        // Inner Loop: Starting point 'i' se aage substring ko badhao
        for (int j = i; j < n; j++) {

            // Agar character pehle se visited hai, toh substring
invalid ho gayi
            if (visited[s[j]] == true) {
                break; // Is loop se bahar niklo, agla starting point
try karo
            }

            // Agar naya character hai, toh use visited mark karo
            visited[s[j]] = true;

            // Ab tak ki length nikal kar max_len ko update karo
            max_len = max(max_len, j - i + 1);
        }
    }

    return max_len;
```

```
}

int main() {
    string s = "pwwkew";
    cout << "String: " << s << endl;
    cout << "Longest Substring Length (Brute Force): " <<
lengthOfLongestSubstringBrute(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N^2)$ — Yahan N string ki length hai. Kyunki humne nested loops use kiye hain, jahan har ek starting point ke liye hum aage tak travel kar rahe hain. Worst case mein (jaise `"abcdefg"` jahan koi character repeat na ho), inner loop poora chalega, jo time ko $O(N^2)$ par le jayega.
- **Space Complexity:** $O(1)$ — Humne 256 size ka ek fixed boolean array banaya hai. Kyunki character set fixed hai (ASCII values 256 hi hoti hain), space input ke badhne se nahi badhegi. Isliye yeh constant auxiliary space hai.

💡 Interview Tip

Jab interviewer tumse kahe ki is $O(N^2)$ ko aur optimize karo, toh unhe batana: *"Sir, brute force mein jab humein repeat character milta hai, toh hum* `i` *ko sirf ek step aage badhate hain aur dobara shuruat se check karte hain. Is repetitive kaam ko bachane ke liye hum* ***Sliding Window (Two Pointer)*** *approach use kar sakte hain, jahan* `i` *ko direct repeat hone waale character ke agle index par jump karwa diya jata hai, jisse time ghatkar pure* $O(N)$ *ho jata hai!"*

Bhai, brute force mein hum baar-baar piche jaakar naye sire se check kar rahe the, jisse time $O(N^2)$ ja raha tha. Ab seekhte hain iska sabse shaandar aur optimized tarika—**Sliding Window (Two Pointers) with Direct Jump Optimization**.

Is approach se hum poori string ko sirf **ek baar** scan karenge aur pure $O(N)$ **Time Complexity** mein answer nikal lenge!

🧠 Intuition: Sliding Window Aur Fast Jump Kya Hai?

Brute force ki sabse badi galti yeh thi ki jab humein koi repeat character milta tha, toh hum poori window ko collapse karke `left` pointer ko bas ek kadam aage badhate the.

Optimized approach mein hum sochte hain: **"Agar koi character repeat hua hai, toh uske pehle waale hisse ko dubara check karne ka koi matlab hi nahi hai."** Hum ek window banate hain do pointers se: `left` aur `right`. `right` pointer window ko aage badhata hai, aur hum ek map/array mein har character ka **aakhiri dekha gaya index (last seen index)** yaad rakhte hain.

Jaise ही `right` pointer par koi aisa character aata hai jo hum pehle dekh chuke hain, toh hum `left` pointer ko ek-ek kadam khiskane ke bajaye **seedha jump karwakar us repeat character ke agle index par le aate hain!**

🔍 Detailed Dry Run (Example: `s = "pwwkew"`)

Hum ek `hash_map` ya `vector<int>` lenge (size 256, initialized with -1) jo characters ka index store karega.
Initial State: `left = 0`, `max_len = 0`

- **`right = 0`** (`s[right] = 'p'`):
- `'p'` ka pichla index kya hai? `-1` (Naya hai).
- `left` apni jagah rahega: `max(0, -1 + 1) = 0`.
- `max_len = max(0, 0 - 0 + 1) = 1`.
- Map mein daal diya: `map['p'] = 0`. Window: `[p]`
- **`right = 1`** (`s[right] = 'w'`):
- `'w'` ka pichla index: `-1` (Naya hai).
- `left` apni jagah: `max(0, -1 + 1) = 0`.
- `max_len = max(1, 1 - 0 + 1) = 2`.
- Map mein daal diya: `map['w'] = 1`. Window: `[p, w]`
- **`right = 2`** (`s[right] = 'w'`):

- `'w'` ka pichla index mila: `1` (**Repeat character!**).
- `left` ne direct jump maari: `max(0, 1 + 1) = 2`. (Ab left seedha index 2 par aa gaya).
- `max_len = max(2, 2 - 2 + 1) = 2`.
- Map update kiya: `map['w'] = 2`. Window: `[w]`
- **`right = 3` (`s[right] = 'k'`):**
- `'k'` ka pichla index: `-1`.
- `left` abhi `2` par hai: `max(2, -1 + 1) = 2`.
- `max_len = max(2, 3 - 2 + 1) = 2`.
- Map mein daal diya: `map['k'] = 3`. Window: `[w, k]`
- **`right = 4` (`s[right] = 'e'`):**
- `'e'` ka pichla index: `-1`.
- `left` abhi `2` par hai: `max(2, -1 + 1) = 2`.
- `max_len = max(2, 4 - 2 + 1) = 3`. (Naya high score! Substring: `"wke"`)
- Map mein daal diya: `map['e'] = 4`. Window: `[w, k, e]`
- **`right = 5` (`s[right] = 'w'`):**
- `'w'` ka pichla index mila: `2`.
- `left` ne fir jump maari: `max(2, 2 + 1) = 3`.
- `max_len = max(3, 5 - 3 + 1) = 3`.
- Map update kiya: `map['w'] = 5`. Window: `[k, e, w]`

Loop khatam! Final Max Length = **3**.

💻 C++ Code (Most Optimized $O(N)$)

Yahan iska sabse efficient implemention hai jo Kisi bhi coding platform par 100% beats dega:

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstringOptimized(string s) {
    int n = s.length();
    int max_len = 0;

    // 256 size ka vector jo har character ka 'last seen index' store
karega
    // Shuruat mein sabko -1 se bhar diya
    vector<int> last_seen(256, -1);

    int left = 0; // Window ka starting point

    // 'right' pointer poori string par aage badhega (Single Pass)
    for (int right = 0; right < n; right++) {
        char current_char = s[right];

        // Agar character pehle dikh chuka hai aur woh hamari current
window ke andar hai,
        // toh left pointer ko direct jump karwao uske agle index par
        if (last_seen[current_char] != -1) {
            left = max(left, last_seen[current_char] + 1);
        }

        // Current character ka index map mein update karo
        last_seen[current_char] = right;

        // Window ki current length nikal kar max_len update karo
        max_len = max(max_len, right - left + 1);
    }

    return max_len;
}
```

```
int main() {
    string s = "pwwkew";
    cout << "Original String: " << s << endl;
    cout << "Optimized Max Substring Length: " <<
lengthOfLongestSubstringOptimized(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N)$ — Kyunki `right` pointer index `0` se lekar `n-1` tak sirf ek hi baar travel karta hai. `left` pointer bina kisi piche waale loop ke direct jumps marta hai. Isliye yeh pure linear time mein poora ho jata hai.
- **Space Complexity:** $O(1)$ — Humne sirf 256 size ka ek fixed integer vector liya hai, jo input string ki length par depend nahi karta (chahe string 10 characters ki ho ya 10 lakh ki, array size hamesha 256 hi rahegi).

Bhai, yeh sliding window ka logic clear hua? Agar tum chaho toh hum is concept ko kisi aur sliding window problem (jaise "Minimum Window Substring") se connect karke aur achhe se samajh sakte hain!

Bhai, pichle turn par jo maine direct jump waala tarika bataya tha, wahi iska **absolute optimal solution** hai. Lekin is approach mein ek aisi baareeki (hidden trap) hai jahan 90% log interview mein galti kar baithte hain!

Chalo us trap ko samajhte hain aur ek aise solid udaharan (`s = "abba"`) ke saath **microscopic dry run** karte hain, jise dekh kar interviewer turant samajh jayega ki aapka logic kitna gehra hai.

🚨 The Hidden Trap: `max(left, last_seen[current_char] + 1)` Kyun?

Kuch log sochte hain ki jab bhi koi repeat character mile, toh `left` pointer ko bina soche `last_seen[current_char] + 1` par kooda do. Lekin yeh galat ho jata hai!

Maan lo string hai **`"abba"`**:

1. `right` pahuncha dusre `'b'` par (index 2). Usne dekha pichla `'b'` index 1 par tha, toh `left` kood kar aa gaya index 2 par. (Yahan tak sab sahi hai).
2. Ab `right` pahuncha aakhiri `'a'` par (index 3). Usne dekha pichla `'a'` toh index 0 par tha! Agar hum bina soche jump maarenge, toh `left` piche ki taraf kood kar **index 1** par chala jayega!
3. Isse hamari window piche khisak jayegi aur answer galat ho jayega. Isiliye hum `max(left, ...)` lagate hain taaki `left` pointer **kabhi piche na jaaye**, hamesha aage hi badhe.

📊 Microscopic Dry Run (Example: `s = "abba"`)

Hum ek `last_seen` array rakhenge jisme saare characters ka pichla index save hoga (shuruat mein sab `-1`).

Right Index	Character	Pichla Index (`last_seen`)	Left ki nayi value: `max(left, last_seen + 1)`	Current Window	Window Length (`right - left + 1`)	Max Length (`max_len`)
Shuruat	-	-	`left = 0`	-	-	`max_len = 0`
`right = 0`	`'a'`	`-1`	`max(0, -1 + 1) = 0`	`"a"`	$0 - 0 + 1 = 1$	**1**
`right = 1`	`'b'`	`-1`	`max(0, -1 + 1) = 0`	`"ab"`	$1 - 0 + 1 = 2$	**2**
`right = 2`	`'b'`	`1` (Repeat!)	`max(0, 1 + 1) = 2` *(Left aage badha)*	`"b"`	$2 - 2 + 1 = 1$	2

`right = 3`	`'a'`	`0` (Repeat!)	`max(2, 0 + 1) = 2` *(Left piche nahi gaya!)*	`"ba"`	$3-2+1=2$	2

Final Answer = 2 (Substrings: `"ab"` ya `"ba"`)

💻 Production-Grade C++ Code (With Detailed Comments)

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstringAbsoluteOptimal(string s) {
    int n = s.length();
    int max_len = 0;

    // ASCII characters 256 hote hain, sabka pichla seen index -1 set
kiya
    vector<int> last_seen(256, -1);

    int left = 0; // Window ka baayein wala hissa

    // 'right' pointer window ko aage badhayega
    for (int right = 0; right < n; right++) {
        char ch = s[right];

        // Agar character pehle aa chuka hai
        if (last_seen[ch] != -1) {
            // TRAP PROTECTION: left ko sirf tabhi badhao agar pichla
index
            // current window ke andar ya aage ho. Piche nahi koodna
hai!
            left = max(left, last_seen[ch] + 1);
        }

        // Current character ka naya index update karo
        last_seen[ch] = right;

        // Har step par window ki size nikal kar max_len update karo
        max_len = max(max_len, right - left + 1);
    }

    return max_len;
```

```
}

int main() {
    string s = "abba";
    cout << "String: " << s << endl;
    cout << "Absolute Optimal Max Length: " <<
lengthOfLongestSubstringAbsoluteOptimal(s) << endl;
    return 0;
}
```

⏱️ Yeh Sabse Optimal Kyun Hai? (The Final Verdict)

1. **Time Complexity:** $O(N)$ — Puri string par sirf **ek single loop** chal raha hai. `right` pointer har character ko sirf ek baar touch karta hai, aur `left` pointer bina kisi inner loop ke seedha O(1) time mein jump marta hai.
2. **Space Complexity:** $O(1)$ — Humne jo `last_seen` vector banaya hai uski size `256` fixed hai. Input string chahe 10 characters ki ho ya 10 million characters ki, memory hamesha 256 integers ki hi rahegi (Constant Space).

Bhai, is `max(left, last_seen[ch] + 1)` waale point ko dhyan se samajh lena, kyunki standard sliding window (jisme set use karke left ko ek-ek karke badhate hain) usme $O(2N)$ time lagta hai, jabki is direct jump approach mein pure $O(N)$ lagta hai!

Bhai, kisi bhi coding interview mein solution batane ke baad, interviewer ka agla sabse bada hathiyar hota hai: **"Edge Cases"**. Agar aapne code likhne se pehle hi khud se edge cases identify kar liye, toh aapka impression 10x behtar ho jata hai.

"Longest Substring Without Repeating Characters" problem ke liye **5 sabse critical edge cases** aur unka behavior ye hain:

---

1. Empty String (`s = ""`)

- **Snafu/Dikkat:** Agar string khali hai, toh hamara code crash nahi hona chahiye.
- **Code Behavior:** Hamare optimized solution mein `n = s.length()` yaani `0` hoga. Loop `right < 0` check karega aur chalega hi nahi. Function direct `max_len = 0` return kar dega.

- **Expected Output:** `0`

2. All Characters are Identical (`s = "bbbbbbb"`)

- **Snafu/Dikkat:** Har ek agla character pichle wale ka repetition hai.
- **Code Behavior:** * `right = 0` par `"b"` dekha, `max_len = 1`.
- `right = 1` par fir `"b"` dekha, jo index 0 par tha. `left` jump karke `0 + 1 = 1` par aa gaya. Window fir se 1 size ki ho gayi.
- Poore loop mein `left` aur `right` saath-saath chalenge, aur window ki size kabhi 1 se badi nahi hogi.
- **Expected Output:** `1`

3. All Characters are Unique (`s = "abcdefg"`)

- **Snafu/Dikkat:** Poori string mein koi bhi character repeat nahi ho raha.
- **Code Behavior:** `last_seen[ch]` hamesha `-1` milega. `left` pointer hamesha `0` par hi betha rahega aur `right` pointer end tak chala jayega. Window expand hoti jayegi aur final answer string ki total length ke barabar ho jayega.
- **Expected Output:** `7` (yaani N)

4. Spaces and Special Characters (`s = "a b c d !@#$")

- **Snafu/Dikkat:** Interviewer check karna chahta hai ki aapka hashmap ya frequency array sirf alphabets (`a-z`) jhel sakta hai ya baki characters bhi.
- **Code Behavior:** Kyunki humne `vector<int> last_seen(256, -1)` liya hai, ye poore **ASCII table (0-255)** ko cover karta hai. Isme space (`' '`), exclamation (`'!'`), `@`, `#` sabki apni unique index position hoti hai, isliye code bina kisi error ke perfectly kaam karega.
- **Expected Output:** Sahi unique length nikal aayegi.

5. Repetition Outside the Current Window (`s = "abba"` ya `s = "tmmzuxt"`)

- **Snafu/Dikkat:** Ye wahi famous logical trap hai jo maine pichle dry run mein bataya tha. Yahan character repeat toh hota hai, par uska pichla occurrence hamari active window se bahar (piche) chala jata hai.

- **Code Behavior:** * `"abba"` mein aakhiri `'a'` (index 3) par jab hum pahuche, toh pichla `'a'` index 0 par tha.
- Lekin hamara `left` pointer tab tak index 2 par pahunch chuka tha.
- `left = max(left, last_seen['a'] + 1)` yaani `max(2, 0 + 1)` ke logic se `left` apni sahi jagah (index 2) par hi barkarar rahega, piche nahi koodega.
- **Expected Output:** `2`

💡 Interview Tip

Jab code complete ho jaye, toh interviewer ke bolne se pehle khud bolna:

> *"Sir, let's verify this code against some edge cases like an empty string, a string with all unique characters, and strings with repeating patterns outside the window like 'abba'..."* Ye sunte hi interviewer samajh jayega ki aap sirf code rat kar nahi aaye ho, aapko test cases ki solid samajh hai!

Bhai, LeetCode aur GeeksforGeeks ka ek aur classic question—**"Smallest Distict Window"** (ya "Smallest window containing all distinct characters").

Is problem ka brute force tarika dimaag mein bithaana isliye zaroori hai, kyunki jab tum iske baad iske optimized version (Sliding Window) par jaoge, toh tumhe samajh aayega ki hum kaunsa faltu kaam bacha rahe hain.

Chalo iska detailed brute force solution, poori intuition aur dry run ke saath samajhte hain.

🎯 Problem Statement Kya Hai?

Aapko ek string di gayi hai. Aapko ek aisi **sabse chhoti substring (window)** dhoondni hai jisme original string ke **saare unique/distinct characters** kam se kam ek baar zaroor maujood hon.

- **Example:** `s = "aabcb"`
- **Total Distinct Characters:** `'a'`, `'b'`, `'c'` (Total = 3 unique characters)
- **Output:** `3` (Kyunki substring `"abc"` ki length 3 hai aur isme teeno unique characters maujood hain).

🧠 Brute Force Intuition: "Check Every Substring"

Brute force ka seedha matlab hai: Poori string mein jitni bhi substrings ban sakti hain, un sabko ek-ek karke check karo aur dekho ki kis substring mein saare unique characters mil rahe hain. Jo sabse chhoti valid substring hogi, wahi hamara answer hogi.

Algorithm Steps:

1. **Total Unique Chars Ginho:** Sabse pehle poori string par ek loop chalao aur pata karo ki total kitne unique characters hain (Hum ek `set` ya `frequency array` ka use karke yeh count nikal sakte hain). Let's call it `total_distinct`.
2. **Saari Substrings Generate Karo:** Do nested loops lagao:

- **Outer Loop (`i`):** Substring ka starting point decide karega.
- **Inner Loop (`j`):** Substring ka ending point decide karega aur window ko aage badhaega.

3. **Unique Chars Track Karo:** Inner loop ke andar ek naya `set` ya array rakho jo current substring ke unique characters ko track karega.
4. **Condition Check:** Jaise hi current substring ke unique characters ka count `total_distinct` ☒☒ barabar ho jaye, iska matlab humein ek valid window mil gayi hai!
5. **Update Minimum Length:** Is window ki length (`j - i + 1`) nikal lo aur hamare `min_len` variable ko update kar do. Iske baad inner loop ko **break** kar do (kyunki `j` ko aur aage badhane se window badi hi hogi, choti nahi).

🔍 Detailed Dry Run (Example: `s = "aabcb"`)

- **Step 1:** Poori string ka unique count nikala → Set `{'a', 'b', 'c'}` bana. `total_distinct = 3`.
- Initial State: `min_len = INT_MAX` (kois bhi bada number)

Outer Loop `i = 0` (Starting with first 'a')

- `j = 0` (`s[0] = 'a'`): Substring = `"a"`. Unique chars = `{'a'}` (Count = 1). Valid nahi hai.
- `j = 1` (`s[1] = 'a'`): Substring = `"aa"`. Unique chars = `{'a'}` (Count = 1). Valid nahi hai.

- `j = 2` (`s[2] = 'b'`): Substring = `"aab"`. Unique chars = `{'a', 'b'}` (Count = 2). Valid nahi hai.
- `j = 3` (`s[3] = 'c'`): Substring = `"aabc"`. Unique chars = `{'a', 'b', 'c'}` (Count = 3). **Valid Mila!** * Length = $3 - 0 + 1 = 4$.
- `min_len = min(INT_MAX, 4) = 4`. Inner loop break!

Outer Loop `i = 1` (Starting with second 'a')

- `j = 1` (`s[1] = 'a'`): Substring = `"a"`. Unique count = 1.
- `j = 2` (`s[2] = 'b'`): Substring = `"ab"`. Unique count = 2.
- `j = 3` (`s[3] = 'c'`): Substring = `"abc"`. Unique count = 3. **Valid Mila!**
- Length = $3 - 1 + 1 = 3$.
- `min_len = min(4, 3) = 3`. Inner loop break!

Outer Loop `i = 2` (Starting with 'b')

- `j = 2` (`s[2] = 'b'`): Substring = `"b"`. Unique count = 1.
- `j = 3` (`s[3] = 'c'`): Substring = `"bc"`. Unique count = 2.
- `j = 4` (`s[4] = 'b'`): Substring = `"bcb"`. Unique count = 2. Loop ☒☒☒☒.

(Baki loops bhi aise hi chalenge par kisi mein bhi teeno unique chars nahi milenge).

Final Answer = 3

💻 C++ Code (Brute Force Solution)

```cpp
#include <iostream>
#include <string>
#include <unordered_set>
#include <algorithm>
#include <climits>
using namespace std;

int smallestDistinctWindowBrute(string s) {
    int n = s.length();
    if (n == 0) return 0;

    // Step 1: Poori string mein total unique characters kitne hain pata
karo
    unordered_set<char> total_distinct_set;
    for (char ch : s) {
        total_distinct_set.insert(ch);
    }
    int total_distinct = total_distinct_set.size();

    int min_len = INT_MAX;

    // Step 2: Outer loop starting point ke liye
    for (int i = 0; i < n; i++) {
        unordered_set<char> current_window_set;

        // Inner loop ending point ke liye (Window expansion)
        for (int j = i; j < n; j++) {
            current_window_set.insert(s[j]);

            // Step 3: Agar current window mein saare distinct
characters aa gaye hain
            if (current_window_set.size() == total_distinct) {
                int current_len = j - i + 1;
                min_len = min(min_len, current_len);

                // Break kyunki is starting point 'i' se iske aage
                // badhne par window ki length sirf badhegi, choti nahi
```

```
hogi
                break;
            }
        }
    }

    return min_len;
}

int main() {
    string s = "aabcb";
    cout << "Original String: " << s << endl;
    cout << "Smallest Distinct Window Length: " <<
smallestDistinctWindowBrute(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N^2)$ — Humne nested loops chalaye hain. Outer loop N baar chalta hai aur inner loop bhi worst case mein $O(N)$ baar chalta hai. Har step par set mein insertion $O(1)$ average leta hai. Isliye total time complexity $O(N^2)$ ho jati hai. Agar string ki length 10^5 ho gayi, toh ye approach TLE (Time Limit Exceeded) de degi.
- **Space Complexity:** `O(K)` ya `O(1)` — Jahan K total unique characters ki sankhya hai. Kyunki characters fixed hote hain (ASCII 256 max), isliye set max 256 elements hi store karega, jo ki constant space hai.

Bhai, brute force ki sabse badi kharabi pakadte hi iska optimized solution dimaag mein chamakta hai. Chalo pehle main tumhe batata hoon ki bade-bade competitive programmers aur engineers **is approach tak pahunche kaise (How to think)**, aur fir iska makkhan jaisa logic aur dry run dekhte hain.

🧠 Humne Kaise Socha? (The Thought Process)

Brute force mein jab hum `s = "aabcb"` par kaam kar rahe the, toh dhyan se dekho hum kya faltu mehnat kar rahe the:

1. Jab `i = 0` tha, toh humne aage badhte hue `"aabc"` dhoondha (index 0 se 3). Yeh valid window thi.
2. Agle step mein jab `i = 1` hua, toh humne index 0 wale `'a'` ko choda, aur **dubara naye sire se** index 1 se aage ginti shuru kar di (`j = 1, 2, 3...`).

> **Dimag ki Batti Jali 💡 :** > Bhai, agar humne index 0 se 3 tak ki window `"aabc"` check kar li hai, aur ab hum index 1 par ja rahe hain, toh humein poori window dubara thodi na banani hai! Humein bas purani window mein se index 0 wale akshar (`'a'`) ko **hata dena hai**.
> Nayi window apne aap `"abc"` ban jayegi! Hum bina kisi naye loop ke, sirf purani window ki information ko use karke aage badh sakte hain. Isko kehte hain **Sliding Window (Two Pointers)** technique.

🚀 Optimized Approach: Sliding Window ($O(N)$)

Hum do pointers lenge: `left = 0` aur `right = 0`.

- **`right` pointer** window ko aage badhaega (expand karega) jab tak humein saare unique characters na mil jayein.
- **`left` pointer** window ko piche se chota karega (shrink karega) taaki hum sabse *chhoti* valid window dhoond sakein.

Hum ek frequency array (ya map) rakhenge jo track karega ki hamari current window mein kaun sa character kitni baar aaya hai.

📊 Detailed Dry Run (Example: `s = "aabcb"`)

- **Total Distinct Characters in String:** `'a'`, `'b'`, `'c'` $\rightarrow$ `total_distinct = 3`.
- **Variables:** `left = 0`, `min_len = INT_MAX`, `distinct_in_window = 0`, `freq` array size 256 (sab 0).

Right	Char	Freq Update	Distinct in Window	Condition: `distinct_in_window == total_distinct`	Left (Shrinking Step) & `min_len` Update
`0`	`'a'`	`freq['a'] = 1`	1	No (1 != 3)	-
`1`	`'a'`	`freq['a'] = 2`	1	No (1 != 3)	-
`2`	`'b'`	`freq['b'] = 1`	2	No (2 != 3)	-
`3`	`'c'`	`freq['c'] = 1`	**3**	**Yes! (3 == 3)**	

Current Window: `"aabc"` | **Valid Window Mili!**

1. `min_len = min(INT_MAX, 3-0+1) = 4`

2. `left=0` (`'a'`) ko hatao → `freq['a']` hua 1.

3. `left` badh kar `1` hua.

Window abhi bhi valid hai (`"abc"`)!

4. `min_len = min(4, 3-1+1) = 3`.

5. `left=1` (`'a'`) ko hatao → `freq['a']` hua 0, `distinct_in_window` ghat kar 2 hua.

6. `left` badh kar `2` hua. Loop Break! |
| `4` | `'b'` | `freq['b'] = 2` | 2 | No (2 != 3) | Loop ends kyunki string khatam ho gayi. |

Final Answer = 3 (Window thi `"abc"`)

💻 Production-Grade C++ Code

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <climits>
using namespace std;

int smallestDistinctWindowOptimized(string s) {
    int n = s.length();
    if (n == 0) return 0;

    // Step 1: Poori string ke unique characters count karo
    vector<bool> visited(256, false);
    int total_distinct = 0;
    for (char ch : s) {
        if (!visited[ch]) {
            visited[ch] = true;
            total_distinct++;
        }
    }

    // Step 2: Sliding Window setup
    vector<int> freq(256, 0);
    int left = 0, distinct_in_window = 0;
    int min_len = INT_MAX;

    // Right pointer string ko scan karega
    for (int right = 0; right < n; right++) {
        char right_char = s[right];

        // Agar ye character window mein pehli baar aa raha hai
        if (freq[right_char] == 0) {
            distinct_in_window++;
        }
        freq[right_char]++;

        // Step 3: Jab window valid ho jaye (saare distinct chars mil
```

```
jayein)
        while (distinct_in_window == total_distinct) {
            // Answer ko update karo
            int current_window_len = right - left + 1;
            min_len = min(min_len, current_window_len);

            // Ab left se window ko chota (shrink) karo
            char left_char = s[left];
            freq[left_char]--;

            // Agar freq 0 ho gayi, matlab ye character window se poora
bahar nikal gaya
            if (freq[left_char] == 0) {
                distinct_in_window--;
            }
            left++; // Left pointer ko aage badhao
        }
    }

    return min_len;
}

int main() {
    string s = "aabcb";
    cout << "Original String: " << s << endl;
    cout << "Smallest Distinct Window Length (Optimized): "
         << smallestDistinctWindowOptimized(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N)$ — Tumhe code mein `while` loop andar dikh raha hoga, par dhyan se socho: `right` pointer index `0` se `N-1` tak jata hai (N steps). Aur `left` pointer bhi poore code mein maximum `0` se `N-1` tak hi aage badh sakta hai (N steps). Dono

pointers milkar puri string ko maximum 2 baar touch karte hain. Isliye time complexity $O(2N) \approx O(N)$ ho jati hai, jo ki super fast hai!

- **Space Complexity:** $O(1)$ — Humne `visited` aur `freq` arrays banaye hain jinki size `256` fixed hai (ASCII characters). Ye input string ki length par depend nahi karti, isliye space constant hai.

Bhai, "Smallest Distinct Window" problem mein sliding window lagate waqt edge cases check karna bohot zaroori hai, kyunki yahan humein do pointers (`left` aur `right`) ko ek saath coordinate karna padta hai. Agar logic thoda bhi hila, toh code ya toh infinite loop mein phans jayega ya fir galat window length de dega.

Chalo iske **5 sabse critical edge cases** aur unka behavior samajhte hain:

---

1. Empty String (`s = ""`)

- **Trap/Snafu:** Agar string mein kuch hai hi nahi, toh unique characters 0 honge aur code ko turant rukna chahiye.
- **Code Behavior:** Code ke shuru mein hi check laga hai `if (n == 0) return 0;`. Yeh bina loop chalaye safe exit kar jata hai, jisse run-time error (segmentation fault) ka koi darr nahi rehta.
- **Expected Output:** `0`

2. Single Character Repeating (`s = "aaaaaaa"`)

- **Trap/Snafu:** Puri string mein sirf ek hi unique character hai. Is case mein `while` loop ka condition har baar trigger ho sakta hai.
- **Code Behavior:** * `total_distinct` nikal kar aayega `1`.
- Jaise hi `right = 0` par pehla `'a'` milega, `distinct_in_window` bhi `1` ho jayega.
- `while` loop turant chalega: `min_len` update hoga `1` par, `left` badhkar `1` ho jayega aur `distinct_in_window` wapas `0` ho jayega.
- Agle saare `'a'` par bhi window ki size `1` se badi nahi ho payegi.
- **Expected Output:** `1`

3. All Characters are Already Unique (`s = "abcdef"`)

- **Trap/Snafu:** Poori string ke saare akshar alag-alag hain.

- **Code Behavior:** `total_distinct` hoga `6`. `right` pointer index `0` se lekar `5` tak badhta chala jayega, par tab tak `distinct_in_window` kabhi `6` nahi pahuchega jab tak `right` aakhiri character `'f'` ko touch nahi kar leta. Aakhiri step par `while` loop sirf ek baar chalega aur `min_len` poori string ki length (N) ke barabar set ho jayega.
- **Expected Output:** `6` (yaani total length N)

4. Target Window at the Very End (`s = "aaaaaaabc"`)

- **Trap/Snafu:** Shuruat mein bohot saare repetitive characters hain aur baki ke unique characters bilkul aakhiri mein chhupkar bethe hain.
- **Code Behavior:** * Unique characters hain `'a'`, `'b'`, `'c'` (Total = 3).
- `right` pointer jab tak aakhiri index (`'c'`) par nahi pahunchta, tab tak `while` loop chalega hi nahi.
- Jaise hi `right` aakhiri index par pahunchega, `while` loop active hoga. Ab `left` pointer (jo index 0 par betha tha) tezi se aage khisakte hue saare faltu `'a'` ko window se bahar nikaal dega, jab tak ki active window sirf `"abc"` nahi reh jaati.
- **Expected Output:** `3` (Window: `"abc"`)

5. Spaces, Numbers aur Special Characters (`s = "a 1 @ a"`)

- **Trap/Snafu:** Interviewer check karta hai ki aapka code alphabets ke alawa baki characters ko sahi se handle karta hai ya nahi.
- **Code Behavior:** Kyunki humne size 256 ka frequency array (`vector<int> freq(256, 0)`) liya hai, isme space (`' '`), number (`'1'`), aur symbol (`'@'`) sabki apni ek unique ASCII value hoti hai. Code in sabko alag aur valid unique character treat karega aur bina kisi crash ke perfect window size return karega.
- **Expected Output:** `5` (Kyunki unique characters hain: `'a'`, `' '`, `'1'`, `'@'` ... total 4 unique characters hain, aur unhe cover karne wali sabse choti window `"a 1 @"` ya `" 1 @ a"` hogi jiski length 5 hai spaces ke saath).

Bhai, KMP (Knuth-Morris-Pratt) Algorithm ki jaan hai **LPS (Longest Proper Prefix which is also Suffix)**. Agar tumne LPS nikalna seekh liya, toh string matching ka sabse mushkil sawal bhi tumhare liye bacchon ka khel ban jayega.

Chalo isko bilkul basic se microscopic level tak todkar samajhte hain.

🧠 Pehle Terminology Samajhte Hain

Kisi bhi string (jaise `s = "abc"`) ke liye:

1. **Prefix:** Jo shuruat se shuru ho → `"a"`, `"ab"`, `"abc"`
2. **Proper Prefix:** Poori string ko chhodkar baki saare prefixes → `"a"`, `"ab"` (Poori `"abc"` isme nahi aayegi).
3. **Suffix:** Jo aakhiri mein khatam ho → `"c"`, `"bc"`, `"abc"`

LPS Array Kya Karta Hai?
`lps[i]` humein batata hai ki substring `s[0...i]` (shuruat se lekar index `i` tak) mein sabse lamba aisa kaun sa **Proper Prefix** hai jo uska **Suffix** bhi hai.

💡 LPS Kaise Kaam Karta Hai? (The $O(N)$ Smart Logic)

Agar hum brute force se har substring ke saare prefix-suffix check karenge, toh $O(N^3)$ lag jayega. Isko $O(N)$ mein karne ke liye hum do pointers use karte hain:

- **`len`:** Yeh track karta hai ki abhi tak kitni length ka prefix match ho chuka hai (shuruat mein `0`).
- **`i`:** Yeh string par aage badhta hai (shuruat mein `1`, kyunki `lps[0]` hamesha `0` hota hai).

Do Hi Rules Hain Iske:

1. **Agar `s[i] == s[len]` (Match Mila):** Iska matlab humein apna prefix-suffix match mil raha hai. `len` ko ek badhao (`len++`), use `lps[i]` mein daalo, aur `i++` karke aage badho.
2. **Agar `s[i] != s[len]` (Mismatch Hua):** Yahan log galti karte hain! Agar mismatch hua, toh hum shuruat se check nahi karte.

- Agar `len != 0` hai, toh hum `len` ko piche koodate hain: `len = lps[len - 1]`. (Hum dhoondte hain ki kya koi chota prefix-suffix abhi bhi valid hai?)
- Agar `len == 0` hai, iska matlab koi match nahi mila, toh chupchaap `lps[i] = 0` set karo aur `i++` kar do.

📊 Microscopic Dry Run (Example: `s = "ababaca"`)

Chalo is string ka LPS array khud bante hue dekhte hain.

- Initial State: `lps` array size 7 (sab `0`), `len = 0`, `i = 1`.
- `lps[0]` hamesha `0` hoga kyunki 1 length ki string ka koi proper prefix nahi hota.

`i`	`s[i]`	`len`	`s[len]`	Condition & Action	`lps` Array State	Naya `i` / `len`
1	`'b'`	`0`	`'a'`	`s[1] != s[0]` aur `len == 0`.		

`lps[1] = 0`. | `[0, 0, 0, 0, 0, 0, 0]` | `i = 2`, `len = 0` |
| **2** | `'a'` | `0` | `'a'` | **Match!** `s[2] == s[0]`.

`len++` → 1. `lps[2] = 1`. | `[0, 0, 1, 0, 0, 0, 0]` | `i = 3`, `len = 1` |
| **3** | `'b'` | `1` | `'b'` | **Match!** `s[3] == s[1]`.

`len++` → 2. `lps[3] = 2`. | `[0, 0, 1, 2, 0, 0, 0]` | `i = 4`, `len = 2` |
| **4** | `'a'` | `2` | `'a'` | **Match!** `s[4] == s[2]`.

`len++` → 3. `lps[4] = 3`. | `[0, 0, 1, 2, 3, 0, 0]` | `i = 5`, `len = 3` |
| **5** | `'c'` | `3` | `'b'` | **Mismatch!** `s[5] != s[3]`.

`len != 0` → `len = lps[2] = 1`. | `[0, 0, 1, 2, 3, 0, 0]` | `i = 5`, `len = 1` *(i ruka hua hai)* |
| **5** | `'c'` | `1` | `'b'` | **Mismatch Again!** `s[5] != s[1]`.

`len != 0` → `len = lps[0] = 0`. | `[0, 0, 1, 2, 3, 0, 0]` | `i = 5`, `len = 0` *(i abhi bhi ruka hai)* |
| **5** | `'c'` | `0` | `'a'` | **Mismatch Again!** `s[5] != s[0]`.

`len == 0` → `lps[5] = 0`. | `[0, 0, 1, 2, 3, 0, 0]` | `i = 6`, `len = 0` *(Ab i aage*

badha) |
| **6** | `'a'` | `0` | `'a'` | **Match!** `s[6] == s[0]`.

`len++` → 1. `lps[6] = 1`. | `[0, 0, 1, 2, 3, 0, 1]` | Loop Khatam |

Final LPS Array for `"ababaca"`: `[0, 0, 1, 2, 3, 0, 1]`

💻 Standard C++ Code (LPS Generator)

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

vector<int> computeLPSArray(string pattern) {
    int m = pattern.length();
    vector<int> lps(m, 0); // Initialize with 0

    int len = 0; // Length of the previous longest prefix suffix
    int i = 1;

    // lps[0] is always 0, toh loop index 1 se shuru hoga
    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        }
        else { // (pattern[i] != pattern[len])
            if (len != 0) {
                // TRICKY STEP: len ko piche jump karwao
                len = lps[len - 1];
                // Note: Is step par 'i' aage nahi badhega, agle
iteration mein firse check hoga
            }
            else { // if (len == 0)
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

int main() {
    string pattern = "ababaca";
```

```
    vector<int> lps = computeLPSArray(pattern);

    cout << "Pattern: " << pattern << endl;
    cout << "LPS Array: ";
    for (int x : lps) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(M)$ — Jahan M pattern ki length hai. Aapko lag sakta hai ki `while` loop ke andar jab `len = lps[len-1]` hota hai toh `i` aage nahi badhta, toh time zyada lagna chahiye. Lekin pure code mein `len` jitni baar badhta (`len++`) hai, maximum utni hi baar ghat bhi sakta hai. `i` hamesha aage badhta hai. Dono milkar linear time $O(M)$ mein poora kaam khatam kar dete hain.
- **Space Complexity:** $O(M)$ — Humne pattern ke barabar size ka ek `lps` array banaya hai information store karne ke liye.

⚠️ Critical Edge Cases to Remember

1. **All characters same (`s = "aaaa"`)**: `lps` nikal kar aayega `[0, 1, 2, 3]` . Kyunki har agla akshar pichle pattern ko extend karta chala jayega.
2. **No repeating patterns (`s = "abcdef"`)**: `lps` nikal kar aayega `[0, 0, 0, 0, 0, 0]` . Kyunki koi proper prefix kabhi suffix ke barabar nahi hoga.

Bhai, LeetCode par "Find the Index of the First Occurrence in a String" (jise bohot se log `strStr()` ya `indexOf()` bhi kehte hain) ek behad basic aur foundational problem hai. KMP Algorithm par koodne se pehle iska brute force dimaag mein bithaana zaroori hai.

Chalo poori detail mein samajhte hain ki aakhir ek normal human dimaag is approach ko **kaise sochega**, iska **logic** kya hai, aur ek **microscopic dry run** ke saath code dekhte hain.

🧠 Humne Kaise Socha? (The Intuition)

Maan lo tumhare paas ek badi si kitaab hai jise hum **`haystack`** bolte hain, aur tumhe usme ek chota sa shabd dhoondna hai jise hum **`needle`** bolte hain.

> **Normal Human Approach:**
> Tum sabse pehle kitaab ke pehle akshar ko dekhoge. Agar woh tumhare shabd ke pehle akshar se match ho gaya, toh tum aage ke aksharon ko ek-ek karke milana shuru karoge.
> Lekin agar teesra ya chotha akshar mismatch ho gaya, toh tum kya karoge? Tum niraash nahi hoge! Tum kitaab mein **sirf ek akshar aage khisak jaoge** aur wahan se naye sire se poora shabd dhoondna shuru kar doge.

Yahi bilkul simple, bina kisi hifi algorithm ke sochi gayi approach hamari **Brute Force (Naive String Matching)** approach banti hai.

🛠️ Algorithm Steps: Kaam Kaise Hoga?

Hum do nested loops ka use karenge:

1. **Outer Loop (`i`):** Yeh `haystack` par ek-ek karke aage badhega. Yeh tay karega ki hamari matching shuru kahan se ho rahi hai.

- **Optimization Trick:** Is loop ko pure `haystack` ke end tak chalane ki zaroori nahi hai. Agar `haystack` mein bache hue akshar `needle` ki length se chote hain, toh match milna namumkin hai. Isliye loop sirf `haystack.length() - needle.length()` tak hi chalega.

2. **Inner Loop (`j`):** Yeh `needle` par chalega (index `0` se lekar `needle.length() - 1` tak). Yeh check karega ki starting point `i` se lekar aage ke saare akshar `needle` se match ho rahe hain ya nahi.
3. **Condition Check:**

- Har step par hum check karenge: Kya `haystack[i + j] == needle[j]` hai?
- Agar kisi bhi point par **mismatch** mila, toh inner loop ko turant `break` kar do aur `i` ko ek step aage badhao.
- Agar inner loop poora chal gaya (yaani `j == needle.length()`), iska matlab humein poora match mil gaya hai! Hum turant starting index `i` return kar denge.

📊 Microscopic Dry Run

Maan lo:

- `haystack = "batman"` (Length $N = 6$)
- `needle = "man"` (Length $M = 3$)
- Loop chalega `i = 0` se lekar `6 - 3 = 3` tak.

Iteration 1: `i = 0` (Starting with 'b')

- `j = 0`: `haystack[0+0]` ('b') != `needle[0]` ('m'). **Mismatch!** Loop break.

Iteration 2: `i = 1` (Starting with 'a')

- `j = 0`: `haystack[1+0]` ('a') != `needle[0]` ('m'). **Mismatch!** Loop break.

Iteration 3: `i = 2` (Starting with 't')

- `j = 0`: `haystack[2+0]` ('t') != `needle[0]` ('m'). **Mismatch!** Loop break.

Iteration 4: `i = 3` (Starting with 'm')

- `j = 0`: `haystack[3+0]` ('m') == `needle[0]` ('m'). **Match!** `j` badhkar 1 hua.
- `j = 1`: `haystack[3+1]` ('a') == `needle[1]` ('a'). **Match!** `j` badhkar 2 hua.
- `j = 2`: `haystack[3+2]` ('n') == `needle[2]` ('n'). **Match!** `j` badhkar 3 hua.
- Inner loop khatam hua kyunki `j` ki value `needle` ki length (3) ke barabar ho gayi. Iska matlab perfect match mila!
- Hum turant return kar denge `i = 3`.

💻 Production-Grade C++ Code (Brute Force)

```
#include <iostream>
#include <string>
using namespace std;

int strStrBruteForce(string haystack, string needle) {
    int n = haystack.length();
    int m = needle.length();

    // Edge case: Agar needle khali hai toh standard behaviour 0 return
karna hota hai
    if (m == 0) return 0;

    // Agar needle haystack se badi hai, toh match milna namumkin hai
    if (n < m) return -1;

    // Outer loop: Sirf wahan tak jahan tak needle fit aa sake
    for (int i = 0; i <= n - m; i++) {
        int j;

        // Inner loop: Needle ke ek-ek char ko haystack se match karo
        for (j = 0; j < m; j++) {
            if (haystack[i + j] != needle[j]) {
                break; // Mismatch hote hi is loop se bahar niklo
            }
        }

        // Agar inner loop poora bina break hue chala, toh j ki value m
ke barabar hogi
        if (j == m) {
            return i; // Pehla occurrence mil gaya, index return karo
        }
    }

    return -1; // Agar poori string ghumne par bhi nahi mila
}

int main() {
```

```
    string haystack = "batman";
    string needle = "man";

    int index = strStrBruteForce(haystack, needle);
    cout << "Haystack: " << haystack << ", Needle: " << needle << endl;
    cout << "First occurrence index: " << index << endl;

    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N \times M)$ — Worst case mein (jaise `haystack = "aaaaaaaaaa"`, `needle = "aaab"`), har ek `i` ke liye inner loop lagbhag M baar chalega aur aakhiri character par fail hoga. Is wajah se outer loop ke N steps aur inner loop ke M steps multiply hokar $O(N \times M)$ bana dete hain.
- **Space Complexity:** $O(1)$ — Hum koi bhi extra memory ya data structure (like vectors, maps, tables) use nahi kar rahe hain. Sirf do-teen pointers/variables use ho rahe hain, isliye space constant hai.

⚠️ Interview Ke Liye Critical Edge Cases

1. **Needle is longer than Haystack (`haystack = "abc"`, `needle = "abcdef"`)**: Code ke shuruat mein hi `if (n < m) return -1;` check isko handle kar lega aur segmentation fault se bachayega.
2. **No Match Found (`haystack = "leetcode"`, `needle = "leeto"`)**: Code poora chalne ke baad silently `-1` return kar dega.
3. **Partial Matches (`haystack = "mississippi"`, `needle = "issip"`)**: Brute force isko sahi se handle karta hai kyunki jab `"issis"` par mismatch hoga, toh `i` agle index par shift hokar dubara check karega.

Bhai, brute force mein sabse badi pareshani kya thi? Jab bhi koi akshar mismatch hota tha, toh `haystack` ka pointer (`i`) wapas piche chala jata tha aur hum naye sire se ginti shuru

karte the. Is baar-baar piche jaane (backtracking) ki wajah se time complexity $O(N \times M)$ pahunch jati thi.

Iska absolute optimized version hai **KMP (Knuth-Morris-Pratt) Algorithm**. Yeh algorithm kehta hai: *"Bhai, jo hissa hum pehle hi match kar chuke hain, use dobara check kyun karna? Chalo piche jaane ke bajaye hamesha aage badhein!"* KMP poore kaam ko pure $O(N + M)$ **Time Complexity** mein nipta deta hai, aur isme hamara wahi purana dost kaam aata hai jise humne abhi seekha tha—**LPS (Longest Proper Prefix Suffix) Array**.

🧠 KMP Ka Core Logic: Humne Kaise Socha?

Maan lo tum `haystack` mein `needle` dhoond rahe ho, aur kaafi dur tak characters match ho gaye, par achanak ek jagah mismatch ho gaya.

KMP kehta hai ki `haystack` ke pointer ko touch bhi mat karo (woh hamesha aage hi badhega). Hum sirf `needle` ke pointer (`j`) ko piche koodayenge. Kahan koodayenge? **`lps[j - 1]`** par! Kyunki LPS humein pehle hi bata chuka hai ki is point tak kitna prefix aur suffix aapas mein barabar hain, toh hum un barabar waale aksharon ko dobara match nahi karenge.

📊 Microscopic Dry Run

Chalo ek badhiya example lete hain:

- `haystack = "ababcababac"` (Length $N = 11$)
- `needle = "ababac"` (Length $M = 6$)

Step 1: Needle Ka LPS Array Nikalo

Humne abhi LPS nikalna seekha tha, toh `needle = "ababac"` ka LPS array banta hai:

- Indices: `[0, 1, 2, 3, 4, 5]`
- Chars: `[a, b, a, b, a, c]`
- **LPS:** `[0, 0, 1, 2, 3, 0]`

Step 2: String Matching (Using `i` for haystack and `j` for needle)

Initial State: `i = 0`, `j = 0`

1. **`i = 0, j = 0`:** `haystack[0]` ('a') == `needle[0]` ('a'). Match! → `i++, j++` (`i=1, j=1`)
2. **`i = 1, j = 1`:** `haystack[1]` ('b') == `needle[1]` ('b'). Match! → `i++, j++` (`i=2, j=2`)
3. **`i = 2, j = 2`:** `haystack[2]` ('a') == `needle[2]` ('a'). Match! → `i++, j++` (`i=3, j=3`)
4. **`i = 3, j = 3`:** `haystack[3]` ('b') == `needle[3]` ('b'). Match! → `i++, j++` (`i=4, j=4`)
5. **`i = 4, j = 4`:** `haystack[4]` ('c') != `needle[4]` ('a'). **Mismatch Hua!**

- Kyunki `j != 0` hai, `j` jump karega: `j = lps[j - 1]` → `lps[3] = 2`.
- *Notice:* `i` abhi bhi `4` par hi ruka hua hai!

6. **`i = 4, j = 2`:** `haystack[4]` ('c') != `needle[2]` ('a'). **Fir se Mismatch!**

- `j` fir jump karega: `j = lps[1] = 0`. `i` abhi bhi `4` par hai.

7. **`i = 4, j = 0`:** `haystack[4]` ('c') != `needle[0]` ('a'). **Fir se Mismatch!**

- Ab kyunki `j == 0` ho chuka hai, iska matlab is akshar se koi umeed nahi hai. Chupchaap `i++` kar do. (`i=5, j=0`)

8. **`i = 5, j = 0` se `i = 10, j = 5` tak:**

- Ab `haystack` ke index 5 se lekar 10 tak `"ababac"` poora ka poora match ho jayega!
- Har step par match milne se `i` aur `j` dono badhte rahenge.
- Aakhiri match par `j` ki value badhkar `6` (`needle.length()`) ho jayegi.

Loop ruk jayega kyunki poora match mil gaya!

- **Starting Index:** `i - j` → `11 - 6 = 5`.

💻 Production-Grade C++ Code (KMP Algorithm)

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Helper function: Needle ka LPS array banane ke liye
vector<int> computeLPS(string pattern) {
    int m = pattern.length();
    vector<int> lps(m, 0);
    int len = 0;
    int i = 1;

    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

// Main KMP Function
int strStrKMP(string haystack, string needle) {
    int n = haystack.length();
    int m = needle.length();

    if (m == 0) return 0; // Empty needle edge case
    if (n < m) return -1; // Needle agar badi hai
```

```
    // Step 1: LPS array taiyar karo
    vector<int> lps = computeLPS(needle);

    int i = 0; // Pointer for haystack
    int j = 0; // Pointer for needle

    // Step 2: Linear scan bina haystack pointer ko piche khiskaaye
    while (i < n) {
        if (haystack[i] == needle[j]) {
            i++;
            j++;
        }

        // Agar poori needle match ho gayi
        if (j == m) {
            return i - j; // Starting index return karo
        }
        // Mismatch case
        else if (i < n && haystack[i] != needle[j]) {
            if (j != 0) {
                j = lps[j - 1]; // Smart jump using LPS
            } else {
                i++; // Agar j shuruat mein hi hai, toh haystack ko aage
badhao
            }
        }
    }

    return -1; // No match found
}

int main() {
    string haystack = "ababcababac";
    string needle = "ababac";

    int index = strStrKMP(haystack, needle);
    cout << "First occurrence index (KMP): " << index << endl;
```

```
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N + M)$ — Preprocessing (LPS banane) mein $O(M)$ time lagta hai. Main matching loop mein `i` pointer hamesha sirf aage badhta hai, kabhi piche nahi jata, jo $O(N)$ steps leta hai. Isliye total time complexity linear ho jati hai, yaani $O(N + M)$.
- **Space Complexity:** $O(M)$ — Humne `needle` ki length ke barabar ka ek extra `lps` array banaya hai patterns ko store karne ke liye, isiliye auxiliary space $O(M)$ hogi.

⚠️ Edge Cases Jo KMP Perfect Handle Karta Hai

1. **Repeated Patterns (`haystack = "aaaaa"`, `needle = "aaa"`)**: KMP isme bina fail hue index 0 return karega, aur jab agla match dhoondna ho (agar saare occurrences chahiye hon), toh yeh bina piche koode aage badh sakta hai.
2. **Partial Failures (`haystack = "mississippi"`, `needle = "issip"`)**: Jab `"issis"` par break hoga, toh `j` ko sahi jagah jump karwa kar yeh bina kisi dikkat ke aage badhega.

Bhai, LeetCode 1392 ("Longest Happy Prefix") sunne mein bada ajeeb lagta hai, lekin dhyan se dekhoge toh yeh wahi purana concept hai jo humne KMP algorithm mein seekha tha!

"Happy Prefix" ka matlab hota hai ek aisa non-empty **Prefix** jo string ka **Suffix** bhi ho (lekin poori string ke barabar nahi hona chahiye, yaani proper prefix hona chahiye). Humse pucha hai ki sabse *lamba* (longest) happy prefix dhoodh kar batao.

Chalo iska sabse pehle **Brute Force solution** samajhte hain, ki ek normal developer isko shuruat mein kaise sochega.

🧠 Brute Force Intuition: Humne Kaise Socha? (The Thought Process)

Humse manga hai **Longest** Happy Prefix.
Agar mujhe sabse lamba match chahiye, toh choti lengths (1, 2, 3...) se check karne ka kya fayda? Agar main sabse **badi possible length** se shuru karoon aur niche ki taraf aaoon, toh jo mujhe **pehla match** milega, wahi automatically sabse lamba (longest) hoga!

Algorithm Steps:

1. Maan lo string `s` ki length N hai. Sabse bade happy prefix ki length maximum $N - 1$ ho sakti hai (kyunki poori string khud ka happy prefix nahi hoti).
2. Hum ek loop chalayenge `len` ka, jo $N - 1$ **se shuru hokar** 1 **tak** niche aayega (Decreasing Order).
3. Har `len` ke liye, hum string mein se do hisse kaatenge (substrings nikalenge):

- **Prefix:** Shuruat se lekar `len` tak $\rightarrow$ `s.substr(0, len)`
- **Suffix:** Aakhiri se piche ki taraf `len` tak $\rightarrow$ `s.substr(n - len, len)`

4. Hum dono substrings ko compare karenge. Jaise hi `prefix == suffix` mil jaye, hum wahi se **turant return** kar denge, kyunki wahi hamara longest happy prefix hoga.
5. Agar loop khatam hone tak kuch na mile, toh khali string `""` return kar denge.

📊 Detailed Dry Run (Example: `s = "ababab"`)

String length $N = 6$. Loop chalega `len = 5` se lekar `1` tak.

Iteration 1: `len = 5`

- **Prefix (pehle 5 chars):** `"ababa"`
- **Suffix (aakhiri 5 chars):** `"babab"`
- **Check:** `"ababa" == "babab"` ? **No!** (Match nahi hua, loop aage badha).

Iteration 2: `len = 4`

- **Prefix (pehle 4 chars):** `"abab"`
- **Suffix (aakhiri 4 chars):** `"abab"`
- **Check:** `"abab" == "abab"` ? **Yes! Match Mil Gaya!**
- Kyunki hum bade se chote ki taraf aa rahe the, toh ye `4` length waala match hi sabse lamba hoga. Hum yahi se `"abab"` return kar denge.

Final Output: `"abab"`

💻 C++ Code (Detailed Brute Force)

```cpp
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

string longestHappyPrefixBrute(string s) {
    int n = s.length();

    // Agar string 1 length ki ya khali hai, toh happy prefix ho hi nahi
sakta
    if (n <= 1) return "";

    // Loop sabse badi possible length (n-1) se shuru karke 1 tak jayega
    for (int len = n - 1; len >= 1; len--) {

        // C++ mein substr(starting_index, length) hota hai
        string prefix = s.substr(0, len);
        string suffix = s.substr(n - len, len);

        // Agar prefix aur suffix barabar hain, toh yehi sabse lamba hai
        if (prefix == suffix) {
            return prefix; // Pehla match hi longest hoga
        }
    }

    return ""; // Agar koi happy prefix nahi mila
}

int main() {
    string s = "ababab";
    cout << "Original String: " << s << endl;
    cout << "Longest Happy Prefix (Brute): " <<
longestHappyPrefixBrute(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N^2)$ — Humara loop maximum N baar chal sakta hai. Aur us loop ke andar, hum `s.substr()` ka use kar rahe hain jo internally characters ko copy karta hai (takes $O(N)$ time) aur fir do strings ko compare karne mein bhi worst case mein $O(N)$ time lagta hai. Is wajah se overall time complexity $O(N \times N) = O(N^2)$ ho jati hai.
- *Note:* Agar LeetCode par string ki length 10^5 hui, toh yeh brute force code **TLE (Time Limit Exceeded)** de dega.
- **Space Complexity:** $O(N)$ — Kyunki har iteration mein hum `prefix` aur `suffix` naam ki nayi substrings bana kar memory mein store kar rahe hain, jo string ki length ke proportional space leti hain.

⚠️ Critical Edge Cases

1. **Single Character (`s = "a"`)**: Proper prefix ki length string se choti honi chahiye (< 1 yaani 0). Isliye 1 length ki string ka answer hamesha `""` hota hai. Code ka `if (n <= 1)` check isko safe rakhta hai.
2. **All Characters Same (`s = "aaaa"`)**: $N = 4$. Pehli hi iteration mein `len = 3` par prefix `"aaa"` aur suffix `"aaa"` match ho jayenge, aur output `"aaa"` aayega, jo bilkul sahi hai.
3. **No Happy Prefix (`s = "leetcode"`)**: Poore loop mein kabhi bhi prefix aur suffix match nahi honge, aur end mein safe tarike se `""` return ho jayega.

Bhai, brute force ka logic aur dry run poora clear hua? Agar tum ready ho, toh mujhe batao, fir hum is TLE waale solution ko KMP ke LPS logic se connect karke pure $O(N)$ **linear time** mein solve karne ka tarika dekhenge!

Bhai, brute force mein jo $O(N^2)$ time lag raha kaatne aur compare karne mein, use hum khatam kar sakte hain agar hum dhyan se is problem ki definition ko dekhein.

"Longest Happy Prefix" ka matlab kya hai? *A proper prefix which is also a suffix.* Aur KMP Algorithm ka **LPS (Longest Proper Prefix Suffix) array** exact yahi kaam karta hai!

Chalo samajhte hain ki kaise hum KMP ke logic se is problem ko pure $O(N)$ **Time Complexity** mein niptayenge.

🧠 Thinking Steps: Humne Kaise Socha? (The Logic)

1. **Definition Match:** LPS array ka har ek index `i` humein batata hai ki substring `s[0...i]` ka sabse lamba proper prefix kaun sa hai jo suffix bhi hai.
2. **The Golden Target:** Agar mujhe **poori string** (yaani index `0` se lekar `N-1` tak) ka longest happy prefix chahiye, toh mujhe poore LPS array se matlab nahi hai. Mujhe sirf LPS array ke **sabse aakhiri element (`lps[N-1]`)** se matlab hai!
3. **The Answer:** `lps[N-1]` humein us happy prefix ki **exact length** bata dega. Ek baar length mil gayi, toh hum shuruat se utne characters utha lenge (`s.substr(0, happy_len)`) aur wahi hamara answer hoga.

📊 Microscopic Dry Run (Example: `s = "ababab"`)

String ki length $N = 6$. Hum iska LPS array banayenge.

- `len = 0` (prefix length track karne ke liye)
- `i = 1` (string par aage badhne ke liye)
- `lps[0]` hamesha `0` hota hai.

`i`	`s[i]`	`len`	`s[len]`	Condition & Action	`lps` Array State	Naya `i` / `len`
1	`'b'`	`0`	`'a'`	`s[1] != s[0]` aur `len == 0`.		

`lps[1] = 0`. | `[0, 0, 0, 0, 0, 0]` | `i = 2, len = 0` |
| **2** | `'a'` | `0` | `'a'` | **Match!** `s[2] == s[0]`.

`len++` → 1, `lps[2] = 1`. | `[0, 0, 1, 0, 0, 0]` | `i = 3, len = 1` |
| **3** | `'b'` | `1` | `'b'` | **Match!** `s[3] == s[1]`.

`len++` → 2, `lps[3] = 2`. | `[0, 0, 1, 2, 0, 0]` | `i = 4, len = 2` |
| **4** | `'a'` | `2` | `'a'` | **Match!** `s[4] == s[2]`.

`len++` → 3, `lps[4] = 3`. | `[0, 0, 1, 2, 3, 0]` | `i = 5, len = 3` |
| **5** | `'b'` | `3` | `'b'` | **Match!** `s[5] == s[3]`.

`len++` → 4, `lps[5] = 4`. | `[0, 0, 1, 2, 3, 4]` | Loop Khatam |

Result Extraction:

- Loop khatam hone par hamara final LPS array bana: `[0, 0, 1, 2, 3, 4]`.
- Sabse aakhiri value kya hai? `lps[N-1]` = `lps[5]` = **`4`**.
- Iska matlab longest happy prefix ki length `4` hai.
- Hum `s.substr(0, 4)` nikalenge, jo ki aayega **`"abab"`**.

💻 Production-Grade C++ Code ($O(N)$ Solution)

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

string longestHappyPrefix(string s) {
    int n = s.length();

    // Edge Case: Agar string 1 length ki ya khali hai, toh happy prefix nahi ho sakta
    if (n <= 1) return "";

    // KMP ka standard LPS array setup
    vector<int> lps(n, 0);
    int len = 0; // Length of the previous longest prefix suffix
    int i = 1;

    // Poori string ka LPS array calculate karo
    while (i < n) {
        if (s[i] == s[len]) {
            len++;
            lps[i] = len;
            i++;
        }
        else {
            if (len != 0) {
                // Smart fallback step (Bina i ko piche khiskaaye len ko jump karwao)
                len = lps[len - 1];
            }
            else {
                lps[i] = 0;
                i++;
            }
        }
    }
```

```
    // Aakhiri index ki value hi sabse lambe happy prefix ki length hai
    int longest_len = lps[n - 1];

    // Shuruat se lekar utni length ki substring return kar do
    return s.substr(0, longest_len);
}

int main() {
    string s = "ababab";
    cout << "Original String: " << s << endl;
    cout << "Longest Happy Prefix (Optimized): " <<
longestHappyPrefix(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N)$ — Hum sirf ek baar string par iterate kar rahe hain LPS array banane ke liye. `i` pointer hamesha aage badhta hai aur `len` maximum N baar hi badh ya ghatt sakta hai. String extraction (`substr`) mein bhi sirf $O(N)$ lagta hai. Isliye overall time complexity linear hai, jo 10^5 length ki string ko bhi millisecond mein solve kar degi.
- **Space Complexity:** $O(N)$ — Humne string ke har character ke liye information store karne ke liye size N ka ek `lps` array banaya hai, isiliye space complexity linear hogi.

⚠️ Edge Cases Jo Yeh Handle Karta Hai

1. **No Happy Prefix (`s = "abcd"`)**: `lps` array banega `[0, 0, 0, 0]`. Aakhiri value `0` aayegi, toh function `s.substr(0, 0)` yaani khali string `""` return karega.
2. **All Identical Characters (`s = "aaaa"`)**: `lps` array banega `[0, 1, 2, 3]`. Aakhiri value `3` aayegi, toh answer `"aaa"` milega, jo ki proper prefix hai aur bilkul sahi hai.
3. **Overlapping Patters (`s = "aacaafaacaa"`)**: KMP ka fallback mechanism (`len = lps[len-1]`) complex patterns ke overlap ko bina phasaye perfect track kar leta hai.

Bhai, LeetCode 214 ("Shortest Palindrome") ek aisa sawal hai jo shuruat mein bohot dravna lagta hai, par iska brute force logic itna simple hai ki ek baar samajh gaye toh bologe— *"Bas itni si baat thi!"*

Chalo isko poori detail mein todte hain ki **kaise sochna hai**, **intuition kya hai**, aur **microscopic dry run** ke saath iska code dekhte hain.

🧠 Thinking Steps: Humne Kaise Socha? (The Intuition)

Problem kehti hai ki humein string ke **sirf aur sirf aage (front mein)** characters add karne hain taaki poori string ek palindrome ban jaye. Aur characters bhi kam se kam add karne hain (shortest palindrome chahiye).

Maan lo hamari string hai `s = "abacd"`.

> **Dimag ki Batti Jali 💡:**
> Agar main string ke aage characters jod raha hoon, toh iska matlab string ka jo aakhiri hissa hai, wahi palat kar sabse aage aayega.
> Lekin agar string ka **shuruat ka koi hissa pehle se hi palindrome** ho? Jaise `"abacd"` mein shuruat ka `"aba"` pehle se hi palindrome hai!
> Toh mujhe `"aba"` ke liye kuch bhi aage jodne ki zaroori nahi hai. Mujhe bas bache hue piche ke hisse (`"cd"`) ko ultana (reverse) karna hai aur aage chipka dena hai!
> `"cd"` ka ulta hua `"dc"` . Isko aage lagaya toh bana: `"dc" + "abacd" = "dcabacd"` .
> Dekho, ban gaya na sabse chota palindrome!

Conclusion of Intuition:

Humein poori string mein shuruat se shuru hone waala **sabse lamba palindrome hissa (Longest Palindromic Prefix)** dhoondna hai. Ek baar woh mil gaya, toh bache hue suffix ko reverse karke string ke aage laga dena hai.

🛠️ Brute Force Algorithm Steps

1. String `s` ki length N hai. Hum check karenge ki kya poori string hi palindrome hai.
2. Agar nahi hai, toh hum piche se ek-ek akshar chhodte jayenge aur check karenge. Yaani hum substrings `s[0...i]` ko check karenge, jahan `i` loop mein $N-1$ se lekar 0 tak niche aayega.

3. Jaise hi humein pehli aisi substring mili jo palindrome hai, wahi hamari **Longest Palindromic Prefix** hogi! Hum loop ko wahin rok denge.
4. Us index `i` ke baad jo bhi bacha hua hissa hai (`s[i+1...N-1]`), use ek alag string variable mein nikalenge.
5. Us bache hue hisse ko **reverse** karenge aur original string ke **aage plus (+)** karke return kar denge.

📊 Microscopic Dry Run (Example: `s = "abacd"`)

- Total Length $N = 5$.
- Hum loop chalayenge `i = 4` se lekar `0` tak.

Loop Index `i`	Current Substring `s[0...i]`	Is Palindrome?	Action
`i = 4`	`"abacd"`	**No** (`'a' != 'd'`)	Loop aage badhao.
`i = 3`	`"abac"`	**No** (`'a' != 'c'`)	Loop aage badhao.
`i = 2`	`"aba"`	**Yes!** (`'a'=='a'` , `'b'=='b'`)	**Match Mil Gaya!** Loop ko break karo.

Result Extraction:

- Longest Palindromic Prefix ka ending point mila index `2` par (yaani `"aba"`).
- Bacha hua piche ka hissa (Suffix) kya hai? Index 3 se end tak $\rightarrow$ **`"cd"`** .
- Is suffix ka reverse nikala $\rightarrow$ **`"dc"`** .
- **Final Answer:** `Reverse Suffix + Original String` $\rightarrow$ `"dc" + "abacd"` = **`"dcabacd"`** .

💻 Production-Grade C++ Code (Brute Force)

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

// Helper function: Check karne ke liye ki koi substring palindrome hai
ya nahi
bool isPalindrome(const string& s, int start, int end) {
    while (start < end) {
        if (s[start] != s[end]) {
            return false;
        }
        start++;
        end--;
    }
    return true;
}

string shortestPalindromeBrute(string s) {
    int n = s.length();
    if (n <= 1) return s; // Khali ya 1 length ki string pehle se hi
palindrome hai

    int longest_palindrome_prefix_end = 0;

    // Piche se shuru karke check karo ki sabse lamba palindrome prefix
kahan tak hai
    for (int i = n - 1; i >= 0; i--) {
        if (isPalindrome(s, 0, i)) {
            longest_palindrome_prefix_end = i;
            break; // Pehla milte hi break, kyunki hum bade se chote ki
taraf aa rahe hain
        }
    }

    // Jo hissa palindrome nahi hai (suffix), use nikaalo
    string remaining_suffix = s.substr(longest_palindrome_prefix_end +
```

```
1);

    // Us bache hue hisse ko reverse karo
    string reversed_suffix = remaining_suffix;
    reverse(reversed_suffix.begin(), reversed_suffix.end());

    // Reversed suffix ko original string ke aage jodkar return kar do
    return reversed_suffix + s;
}

int main() {
    string s = "abacd";
    cout << "Original String: " << s << endl;
    cout << "Shortest Palindrome (Brute): " <<
shortestPalindromeBrute(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N^2)$ — Outer loop worst case mein N baar chal sakta hai (agar koi bhi prefix palindrome na mile siwaye pehle akshar ke). Aur har iteration ke andar, hum `isPalindrome` function chala rahe hain jo string ko check karne mein $O(N)$ time leta hai. Is wajah se overall time complexity $O(N \times N) = O(N^2)$ ho jati hai. Agar string ki length 10^5 hui, toh yeh **TLE (Time Limit Exceeded)** de dega.
- **Space Complexity:** $O(N)$ — Humne `remaining_suffix` aur `reversed_suffix` naam ki nayi strings banayi hain jo worst case mein lagbhag string ki length ke barabar space le sakti hain.

⚠️ Critical Edge Cases

1. **Already a Palindrome (`s = "racecar"`)**: `i = 6` par hi pehli baar mein poori string palindrome mil jayegi. `longest_palindrome_prefix_end = 6` ho jayega. `remaining_suffix` khali string `""` banegi, aur output wapas `"racecar"` hi aa jayega.

2. **All Distinct Characters (`s = "abcd"`)**: Poore loop mein koi bhi substring palindrome nahi milegi, jab tak `i = 0` (yaani single character `"a"`) nahi aa jata. `"a"` hamesha palindrome hota hai. Suffix banega `"bcd"`, reverse hoga `"dcb"`, aur answer aayega `"dcbabcd"`.
3. **Single or Empty Character (`s = "a"` ya `""`)**: Code ka shuruat ka base condition `if (n <= 1) return s;` isko handle karke bina kisi loop ke sahi output de dega.

Bhai, brute force mein jo $O(N^2)$ lag raha tha use hatane ke liye humein ek gazab ka dimaag lagana padega. KMP ka **LPS (Longest Proper Prefix Suffix)** array humein batata hai ki kisi string ke shuruat ka kitna hissa uske aakhiri hisse se match karta hai.

Lekin pichle sawal mein humne dekha tha ki humein yahan **Longest Palindromic Prefix** dhoondna hai (yaani string ke shuruat ka sabse lamba hissa jo khud ek palindrome ho).

Ab tum sochoge: "LPS toh prefix-suffix dhoondta hai, yeh palindrome kaise dhoondega?"
Chalo iska ek makkhan jaisa logic bante hue dekhte hain!

🧠 Thinking Steps: Humne Kaise Socha? (The KMP Trick)

Maan lo hamari original string hai `s = "abacd"`.
Agar main is string ko ulta (reverse) kar doon, toh yeh ban jayegi `rev = "dcaba"`.

Ab dhyan se dono ko dekho:

- `s = ` **`aba`** `cd`
- `rev = cd` **`aba`**

> **Dimag ki Batti Jali 💡:**
> Original string ka shuruat ka hissa (`"aba"`) aur reversed string ka aakhiri hissa (`"aba"`) bilkul same hain! Kyunki `"aba"` ek palindrome hai, isiliye ulta karne par bhi woh badla nahi.
> Iska matlab, agar main ek nayi string banaaoon dono ko jodkar:
> `Nayi String = s + '#' + rev` (Yahan `#` ek special character hai taaki dono mix na hon)
> `Nayi String = "abacd#dcaba"`
> Ab agar main is poori nayi string ka **LPS Array** nikaloon, toh iska sabse aakhiri element (`lps.back()`) mujhe kya batayega? Woh batayega ki is poori string ka sabse lamba prefix kaun sa hai jo iska suffix bhi hai!
> Aur iska prefix shuru ho raha hai `s` se, aur suffix khatam ho raha hai `rev` par. Is wajah se `lps` ki aakhiri value humein **Longest Palindromic Prefix ki exact length** de degi!

📊 Microscopic Dry Run (Example: `s = "abacd"`)

1. `s = "abacd"`, `rev = "dcaba"`
2. Combined String `temp = "abacd#dcaba"` (Length = 11)
3. Chalo is `temp` string ka LPS array calculate karte hain:

- Chars: `[a, b, a, c, d, #, d, c, a, b, a]`
- **LPS:** `[0, 0, 1, 0, 0, 0, 0, 0, 1, 2, 3]`

Notice: Sabse aakhiri index (index 10) par value kya aayi? **`3`**.
Iska matlab hamari original string `s` mein starting se **3 length** ka hissa (`"aba"`) sabse lamba palindrome prefix hai!

Result Extraction:

- Longest Palindrome Prefix ki length = `3`.
- Original string `s = "abacd"` mein se bacha hua piche ka hissa (index 3 se end tak) kya hai? → `"cd"`.
- Is bache hue hisse ko reverse karo → `"dc"`.
- **Final Answer:** `reversed_suffix + s` → `"dc" + "abacd"` = **`"dcabacd"`**.

💻 Production-Grade C++ Code ($O(N)$ KMP Solution)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

// Standard KMP LPS Generator Function
vector<int> computeLPS(string str) {
    int m = str.length();
    vector<int> lps(m, 0);
    int len = 0;
    int i = 1;

    while (i < m) {
        if (str[i] == str[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

string shortestPalindromeOptimized(string s) {
    int n = s.length();
    if (n <= 1) return s;

    // Step 1: Original string ka reverse nikalo
    string rev = s;
    reverse(rev.begin(), rev.end());
```

```
    // Step 2: Ek combined string banao beech mein special character '#'
lagakar
    // '#' lagana zaroori hai taaki 's' aur 'rev' ke patterns aapas mein
overlap na karein
    string temp = s + "#" + rev;

    // Step 3: Combined string ka LPS array nikalon
    vector<int> lps = computeLPS(temp);

    // Step 4: LPS array ka aakhiri element hi longest palindromic
prefix ki length hai
    int longest_palindromic_prefix_len = lps.back();

    // Step 5: Jo hissa palindrome nahi hai (suffix), use original
string se nikalo
    string remaining_suffix = s.substr(longest_palindromic_prefix_len);

    // Step 6: Us bache hue hisse ko reverse karo
    string reversed_suffix = remaining_suffix;
    reverse(reversed_suffix.begin(), reversed_suffix.end());

    // Final Shortest Palindrome return karo
    return reversed_suffix + s;
}

int main() {
    string s = "abacd";
    cout << "Original String: " << s << endl;
    cout << "Shortest Palindrome (Optimized $O(N)$): " <<
shortestPalindromeOptimized(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N)$ — String ko reverse karne mein $O(N)$ lagta hai. Combined string `temp` ki length $2N + 1$ hogi. KMP ka LPS array calculation pure linear time mein kaam karta hai, toh `computeLPS` function $O(2N + 1) \approx O(N)$ time lega. Substring nikalna aur concatenate karna bhi $O(N)$ hai. Is wajah se poora code $O(N)$ **linear time** mein solve ho jata hai, jo 10^5 length ke input par bhi bilkul nahi phansega.
- **Space Complexity:** $O(N)$ — Humne `temp` string banayi aur size $2N + 1$ ka ek `lps` vector banaya hai data store karne ke liye, isiliye auxiliary space complexity $O(N)$ hogi.

❓ Ek Chota Sa Sawaal (Just to Be Sure)

Bhai, kya tumhe yeh samajh aaya ki humne beech mein `'#'` character kyun lagaya? Agar hum sirf `s + rev` kar dete bina kisi symbol ke, toh `s = "aaaa"`, `rev = "aaaa"` ke case mein loop galat tarike se overlap ko count kar leta aur window boundaries kharab ho jatein.

KMP ka yeh approach dimaag mein acche se fit baitha?

Bhai, humne KMP algorithm se $O(N)$ toh dhoodh liya, jo interview clear karne ke liye perfect hai. Lekin agar tumse interviewer kahe ki **"Mujhe $O(N)$ Space bhi nahi chahiye, $O(1)$ Auxiliary Space mein karke dikhao"**, tab kaam aati hai DSA ki sabse khun kharaba aur elegant technique: **Two Pointers (String Window Matching)** approach.

Isko bina kisi extra array (jaise LPS) ke, sirf do pointers ghuma kar pure $O(N)$ **Time** aur $O(1)$ **Space** mein kaise solve karte hain, chalo iski poori theory aur microscopic level ka dry run dekhte hain.

🧠 Core Theory: Humne Kaise Socha? (The Logic)

Humara target abhi bhi wahi hai: **Longest Palindromic Prefix** dhoondna (yaani string ke shuruat ka sabse lamba hissa jo palindrome ho).

Maan lo hamari string `s` hai. Hum do pointers lete hain:

- `i = 0` (String ke shuruat mein)
- `j = n - 1` (String ke bilkul aakhiri mein)

Hum `j` ko piche khiskate hain jab tak `s[i] == s[j]` na mile.

> **Lekin Twist Yeh Hai ⚠️:**
> Agar beech mein kahin `s[i] != s[j]` ho gaya, toh iska matlab yeh nahi ki hum har maan lein. Hum `j` ko toh piche khiskate hi rahenge, par saath mein `i` ko wapas `0` par le aayenge!
> Jab `j` pointer poora piche (index 0 tak) ghum chuka hoga, toh `i` pointer jahan tak pahunch paaya hoga, woh humein batayega ki **is boundary ke andar hi hamara sabse lamba palindrome prefix chhupa hai**.

Hum is boundary ko pakadenge, bache hue piche ke hisse ko cut karenge, aur use **Recursion** ki madad se chota karke solve kar lenge. Isme koi extra array nahi banta, isliye space complexity $O(1)$ (excluding recursion stack) ho jati hai!

🛠️ Algorithm Steps (Step-by-Step)

1. Do pointers set karo: `i = 0`, `j = n - 1`.
2. Ek loop chalao jab tak `j >= 0` hai:

- Agar `s[i] == s[j]` milta hai, toh `i++` karo (yaani match aage badhao).
- `j--` toh har step par hoga hi hoga.

3. Jab loop khatam ho jaye, toh check karo:

- Agar `i == n` ho gaya, iska matlab poori string hi pehle se palindrome thi! Kuch jodne ki zaroori nahi, original string return kar do.

4. Agar `i < n` hai, iska matlab index `i` se lekar `n-1` tak ka jo piche ka hissa (suffix) hai, woh palindrome ka hissa nahi ban paaya.
5. Hum us bache hue suffix ko cut karke **reverse** karenge.
6. Aur beech waale hisse (`s.substr(0, i)`) ko dobara isi function mein bhej denge (**Recursive Call**) taaki bacha hua palindrome prefix confirm ho sake.

📊 Microscopic Dry Run (Example: `s = "abacd"`)

- Total Length $N = 5$.
- Loop chalega `j = 4` se lekar `0` tak. `i` shuru hoga `0` se.

Step	`j` (Piche se)	`s[j]`	`i` (Aage se)	`s[i]`	Condition Check	Naya `i` State
1	`4`	`'d'`	`0`	`'a'`	`s[0] != s[4]` ('a' != 'd')	Match nahi hua. `i` wapas `0` par hi rha.
2	`3`	`'c'`	`0`	`'a'`	`s[0] != s[3]` ('a' != 'c')	Match nahi hua. `i` wapas `0` par hi rha.
3	`2`	`'a'`	`0`	`'a'`	`s[0] == s[2]` ('a' == 'a')	**Match!** `i` badhkar `1` hua.
4	`1`	`'b'`	`1`	`'b'`	`s[1] == s[1]` ('b' == 'b')	**Match!** `i` badhkar `2` hua.
5	`0`	`'a'`	`2`	`'a'`	`s[2] == s[0]` ('a' == 'a')	**Match!** `i` badhkar `3` hua.

Loop Khatam! (Kyunki `j` ab `-1` ho gaya).

Boundary Breakdown at the end of loop:

- `i` ki final value kya aayi? **`3`**.
- Iska matlab index `3` se lekar end tak ka hissa (`s.substr(3)`) yaani **`"cd"`** kisi bhi haal mein palindrome prefix ka part nahi hai!
- Hum `"cd"` ko alag nikal kar reverse karenge $\rightarrow$ **`"dc"`**.
- Ab hum index `0` se `3` tak ki substring (`"aba"`) ko dubara recursive call mein bhejenge: `shortestPalindrome("aba")`.
- `"aba"` pehle se palindrome hai, toh woh recursive call wapas `"aba"` hi return karega.

Final Combine Result: `reversed_suffix ("dc") + shortestPalindrome("aba") + remaining_suffix ("cd")`
$\rightarrow$ `"dc" + "aba" + "cd"` = **`"dcabacd"`**.

💻 Production-Grade C++ Code ($O(1)$ Auxiliary Space)

```cpp
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

string shortestPalindrome(string s) {
    int n = s.length();
    if (n <= 1) return s;

    int i = 0;
    // Pointers approach to find the prefix boundary
    for (int j = n - 1; j >= 0; j--) {
        if (s[i] == s[j]) {
            i++;
        }
    }

    // Agar i poore n tak pahunch gaya, matlab poori string hi
palindrome hai
    if (i == n) return s;

    // Suffix nikaalo jo match nahi hua (index i se lekar aakhiri tak)
    string remaining_suffix = s.substr(i);

    // Us remaining suffix ko reverse karo
    string reversed_suffix = remaining_suffix;
    reverse(reversed_suffix.begin(), reversed_suffix.end());

    // Recursion lagao baaki bache hue andar waale hisse par (0 se leke
i tak)
    // Aur makkhan ki tarah combine karke return kar do
    return reversed_suffix + shortestPalindrome(s.substr(0, i)) +
remaining_suffix;
}

int main() {
    string s = "abacd";
```

```
    cout << "Original String: " << s << endl;
    cout << "Shortest Palindrome (Two Pointer Optimized): " <<
shortestPalindrome(s) << endl;
    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N)$ **on Average (Worst Case** $O(N^2)$ **but practically highly optimized)** * Har recursive step mein hum string ka non-palindrome hissa kaat kar chota kar dete hain. Average cases aur random strings mein yeh linear time $O(N)$ mein khatam ho jata hai kyunki boundary bohot tezi se shrink hoti hai.
- *Worst Case Scenario:* Agar string `"aababaea"` jaisi ho jahan baar-baar partial matches milein, toh yeh worst case mein $O(N^2)$ ja sakta hai, par LeetCode par yeh solution saare test cases ko bohot kam runtime mein pass kar deta hai kyunki isme string copies bohot kam hoti hain.
- **Space Complexity:** $O(1)$ **Auxiliary Space** * Humne koi KMP ka LPS array (vector) nahi banaya, na hi koi hash map use kiya. Hum sirf pointers ka khel khel rahe hain.
- *Note:* Agar recursion stack memory ko count karein, toh worst case mein stack ki depth $O(N)$ ho sakti hai, par extra memory consumption ke maamle mein yeh absolute $O(1)$ auxiliary space hai.

Bhai, yeh Two-Pointer aur Recursion ka deadly combination dimaag mein acche se fit baitha? Interviewer ko jab tum pehle Brute, fir KMP ($O(N)$ Space) aur aakhiri mein yeh $O(1)$ Space waala logic doge, toh woh poora fida ho jayega!

Bhai, LeetCode 686 ("Repeated String Match") ek bohot hi shaandar question hai. Shuruat mein lagta hai ki isme string ko infinite times repeat karna padega, par asal mein iske piche ek bohot pyaari mathematics aur limit chhupi hui hai.

Chalo bilkul scratch se samajhte hain ki ek normal human dimaag iska brute force solution **kaise sochega**, iski **mathematical limit** kya hai, aur poore **dry run** ke saath code dekhte hain.

🧠 Humne Kaise Socha? (The Intuition & Math Limit)

Humein do strings di hain: `a` aur `b`. Humein batana hai ki `a` ko **kam se kam kitni baar repeat (multiply)** karein taaki string `b`, us repeated string ka ek substring ban jaye.

Maan lo `a = "abcd"` aur `b = "cdabcdab"`.

> **Basic Logic 💡:**
> Agar `b` ko `a` ke andar fit hona hai, toh repeated `a` ki length kam se kam `b` ki length ke barabar ya usse badi toh honi hi chahiye.
> **Sabse Bada Sawaal: Hum `a` ko maximum kitni baar repeat karke check karenge?**
> Yahan lagti hai ek simple si math trick. Maan lo `a` ki length N hai aur `b` ki length M hai.
>
> 1. **Minimum Repeats:** `b` ko cover karne ke liye humein `a` ko tab tak repeat karna padega jab tak uski length $\geq M$ na ho jaye. Maan lo is point par humne `a` ko K baar repeat kiya.
> 2. **The Maximum Boundary (+1 ya +2 tak kyun?):** > * Aisa ho sakta hai ki `b` bilkul perfectly alignment mein na ho. Woh `a` ke aakhiri hisse se shuru ho rahi ho aur agle `a` ke shuruat ke hisse par khatam ho rahi ho (jaise hamare example mein `"cd"` se shuru ho kar `"ab"` par khatam ho rahi hai).
>
> - Isliye alignment check karne ke liye humein `a` ko **maximum** $K + 2$ baar hi repeat karne ki zaroori hoti hai. Agar $K + 2$ baar repeat karne par bhi `b` match nahi mila, toh poori dunya mein `a` ko chahe jitna repeat kar lo, `b` kabhi uska substring nahi ban payega!

🛠️ Brute Force Algorithm Steps

1. Ek temporary string banao `repeated_a = a` aur ek counter rakho `count = 1`.
2. **Loop 1 (Length match karne ke liye):** Jab tak `repeated_a` ki length `b` ki length se choti hai (`repeated_a.length() < b.length()`), tab tak `a` ko usme jode (`+`) chale jao aur `count++` karte raho.
3. Jaise hi length barabar ya badi ho jaye, check karo: Kya `b` ab `repeated_a` ke andar mil raha hai? (C++ mein `repeated_a.find(b) != string::npos`). Agar mil gaya, toh turant **`count`** return kar do.

4. **Loop 2 (Safe Boundary Check - Max 2 more times):** Agar nahi mila, toh `a` ko ek aur baar jodo, `count++` karo aur firse check karo.
5. Agar ab bhi nahi mila, toh `a` ko ek aakhiri baar aur jodo, `count++` karo aur check karo.
6. Agar iske baad bhi match nahi milta, toh aankh band karke `-1` return kar do, kyunki aage check karna bekaar hai.

📊 Microscopic Dry Run

Maan lo:

- `a = "abcd"` (Length $N = 4$)
- `b = "cdabcdab"` (Length $M = 8$)

Initial State:

- `repeated_a = "abcd"`, `count = 1`

Step 1: Length standard tak le jana

- `repeated_a.length()` (4) < `b.length()` (8) $\rightarrow$ True!
- `repeated_a` ban gaya `"abcdabcd"` (Length = 8)
- `count` badhkar hua **`2`**.
- *Check:* Kya `"cdabcdab"` substring hai `"abcdabcd"` ka? **No!** (Kyunki aakhiri mein `'ab'` missing hai).

Step 2: Boundary Check +1

- `a` ko ek aur baar joda $\rightarrow$ `repeated_a` ban gaya `"abcdabcdabcd"` (Length = 12)
- `count` badhkar hua **`3`**.
- *Check:* Kya `"cdabcdab"` substring hai `"abcdabcdabcd"` ka?
- `"ab` **`cdabcdab`** `cd"` $\rightarrow$ **Yes! Match Mil Gaya!**
- Hum yahi se return kar denge `count = 3`.

💻 Production-Grade C++ Code (Brute Force)

```
#include <iostream>
#include <string>
using namespace std;

int repeatedStringMatchBrute(string a, string b) {
    string repeated_a = a;
    int count = 1;

    // Step 1: Tab tak repeat karo jab tak repeated_a ki length b ke
barabar ya badi na ho jaye
    while (repeated_a.length() < b.length()) {
        repeated_a += a;
        count++;
    }

    // Step 2: Pehla check (Jab length just badi ya barabar hui hai)
    if (repeated_a.find(b) != string::npos) {
        return count;
    }

    // Step 3: Boundary check +1 (Agar b thoda shift hokar overlap kar
rha ho)
    repeated_a += a;
    count++;
    if (repeated_a.find(b) != string::npos) {
        return count;
    }

    // Step 4: Boundary check +2 (Extreme shift case ke liye ek aakhiri
baar check)
    repeated_a += a;
    count++;
    if (repeated_a.find(b) != string::npos) {
        return count;
    }

    // Agar itne ke baad bhi nahi mila toh kabhi nahi milega
```

```
    return -1;
}

int main() {
    string a = "abcd";
    string b = "cdabcdab";

    int result = repeatedStringMatchBrute(a, b);
    cout << "Minimum repeats required: " << result << endl;

    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N \times M)$ **Worst Case** * `while` loop zyada se zyada $\frac{M}{N}$ baar chalega, jo ki string banana mein linear time lega.
- Lekin loop ke baad jo hum `repeated_a.find(b)` use kar rahe hain, woh C++ ka built-in string matching hai. Brute force internal implementation ki wajah se `find` function worst case mein $O(\text{Length of repeated\_a} \times \text{Length of } b)$ leta hai.
- Kyunki `repeated_a` ki max length lagbhag $M + 2N$ tak hi ja sakti hai, isliye overall worst-case time complexity $O(M \times (M + N))$ ya simplified term mein $O(N \times M)$ ho jati hai.
- **Space Complexity:** $O(N + M)$ — Hum ek nayi `repeated_a` string generate kar rahe hain memory mein jiski length maximum $M + 2N$ tak jati hai, isiliye dynamic space complexity linear hogi.

⚠️ Interviewer Ke Saamne Bolne Waale Edge Cases

1. **b is already a substring (`a = "abcdef"`, `b = "cde"`)**: `while` loop chalega hi nahi kyunki length pehle se badi hai. Pehle hi `if` check mein match mil jayega aur `1` return ho jayega.
2. **Single Character String (`a = "a"`, `b = "aaaaa"`)**: `while` loop perfectly 5 baar chalega, length match hote hi `5` return kar dega.

3. **Impossibility Case (`a = "abc"`, `b = "xyz"`)**: Dono checks fail honge aur code silently bina infinite loop mein phase `-1` return kar dega.

Bhai, brute force mein jo hum `.find()` use kar rahe hain, woh internally $O(N \times M)$ le raha hai. Iska **absolute optimized solution** nikalta hai hamare usi purane dost se—**KMP Algorithm (LPS Array)**.

KMP ka use karke hum string matching ko $O(N \times M)$ se hata kar pure $O(N + M)$ **linear time** mein convert kar sakte hain. Chalo iska pura optimization framework dekhte hain ki isme dimaag kaise lagana hai.

🧠 Thinking Steps: Optimization Ka Khel

Pichle brute force se humne ek bohot badhiya cheez seekhi thi: **Maximum Boundary Limitation**.

- Agar `a` ki length N hai aur `b` ki length M hai, toh humein `a` ko kam se kam utni baar repeat karna hai taaki uski length M ke barabar ya badi ho jaye (Maan lo K baar).
- Safe check ke liye hum use maximum $K + 2$ tak hi lekar jaate hain.

KMP Ka Tadka 🍳:
Hum `a` ko utni hi baar repeat karke ek final badi string `repeated_a` bana lenge (maximum $K + 2$ times). Ab is `repeated_a` ke andar `b` ko dhoondne ke liye hum brute force `.find()` nahi chalayenge. Hum chalayenge **KMP Algorithm**!

1. `b` (jo hamara pattern hai) ka **LPS Array** nikalenge.
2. `repeated_a` ke andar `b` ko search karenge.
3. Agar match mil jata hai, toh humen pure KMP scanned index se pata chal jayega ki match kahan par khatam hua. Us ending index ke basis par hum exact `count` calculate kar lenge.

📊 Microscopic Dry Run (Example: `a = "abcd"`, `b = "cdabcdab"`)

- $N = 4, M = 8$
- `b` ki length tak pahunchne ke liye `a` ko minimum 2 baar repeat kiya $\rightarrow$ `"abcdabcd"` (Length 8).

- Alignment shift handle karne ke liye humne 2 baar aur repeat kar diya (Total 4 times) $\rightarrow$ `repeated_a = "abcdabcdabcdabcd"` (Length 16).

Step 1: Pattern (`b = "cdabcdab"`) Ka LPS Array

- Chars: `[c, d, a, b, c, d, a, b]`
- **LPS:** `[0, 0, 0, 0, 1, 2, 3, 4]`

Step 2: KMP Search

Hum `repeated_a` ke andar `b` ko scan karenge.

- `repeated_a = "ab cdabcdab cdabcd"`
- KMP ko exact match milega index `2` se lekar index `9` tak!
- Match ka ending index mila `9`.

Step 3: Exact Count Calculation Formula

Humein poori string (Length 16) ko faltu ginte rehne ki zaroori nahi hai. Agar match index `9` par khatam ho raha hai, iska matlab humne asal mein `repeated_a` ke index `9` tak ka hi hissa use kiya hai.

- Index `9` tak kitne characters cover hue? `9 + 1 = 10` characters.
- `a` ki original length kya thi? `4`.
- Toh actual mein `a` kitni baar repeat hua? $\lceil 10/4 \rceil = 3$ baar!

Humein bina kisi extra loop ke direct answer **`3`** mil gaya.

💻 Production-Grade C++ Code (KMP Optimized)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
using namespace std;

// Standard KMP LPS Generator
vector<int> computeLPS(string pattern) {
    int m = pattern.length();
    vector<int> lps(m, 0);
    int len = 0;
    int i = 1;

    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

int repeatedStringMatchKMP(string a, string b) {
    int n = a.length();
    int m = b.length();

    // Step 1: Pattern 'b' ka LPS array taiyar karo
    vector<int> lps = computeLPS(b);
```

```
    // Step 2: 'a' ko utna repeat karo jo safe boundary (K+2) tak jaye
    string repeated_a = "";
    int repeat_limit = (m / n) + 2;
    for (int i = 0; i < repeat_limit; i++) {
        repeated_a += a;
    }

    // Step 3: KMP Matcher Logic
    int i = 0; // Pointer for repeated_a
    int j = 0; // Pointer for b

    while (i < repeated_a.length()) {
        if (repeated_a[i] == b[j]) {
            i++;
            j++;
        }

        if (j == m) {
            // Match mil gaya! i abhi ending index se ek agge khisak
chuka hai.
            // i characters ko cover karne ke liye 'a' ko kitni baar
poora use karna pada:
            return ceil((double)i / n);
        }
        else if (i < repeated_a.length() && repeated_a[i] != b[j]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }

    return -1; // Agar safe limit tak repeat karne par bhi nahi mila
}

int main() {
```

```
    string a = "abcd";
    string b = "cdabcdab";

    int ans = repeatedStringMatchKMP(a, b);
    cout << "Minimum repeats required (KMP Optimized): " << ans << endl;

    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N + M)$
- `b` ka LPS array banane mein $O(M)$ laga.
- `repeated_a` string banane mein aur use KMP se linear scan karne mein max length $(M + 2N)$ tak hi jana padta hai, jo $O(N + M)$ leta hai.
- Overall time complexity pure quadratic $O(N \times M)$ se drop hokar **linear** $O(N + M)$ par aa gayi.
- **Space Complexity:** $O(N + M)$ — Pattern `b` ke liye size M ka `lps` array aur match check karne ke liye max size $(M + 2N)$ ki `repeated_a` string memory mein store ho rahi hai.

Bhai, KMP se index tracking aur `ceil()` ka combination samajh aya? Yeh linear approach string matching ke top tier optimization mein aati hai!

Bhai, humne KMP ($O(N + M)$ Time aur $O(N + M)$ Space) toh dekh liya. Lekin kya hum isko aur zyada optimize kar sakte hain? **Haan, bilkul!** Agar interviewer tumse kahe ki *"Mujhe KMP ka extra space ya strings ko physically memory mein concatenate/multiply karna hi nahi hai. Mujhe pure* $O(1)$ ***Auxiliary Space*** *aur* $O(N + M)$ ***Time*** *mein karke dikhao"*, tab kaam aati hai hamari **Virtual Indexing (Circular Pointer Simulation)** technique.

Chalo iska microscopic level par logic aur dry run samajhte hain.

🧠 Thinking Steps: Virtual Indexing Ka Khel ($O(1)$ Space)

Humein `a` ko repeat karne ki koi zaroori nahi hai. Hum string `a` ko ek **Circular Loop** ki tarah treat kar sakte hain using the Modulo Operator (`%`).

Maan lo `a = "abcd"`. Iska index `0` par `'a'`, `1` par `'b'`, `2` par `'c'`, aur `3` par `'d'` hai.

- Agar main index `4` par jaana chahoon, toh real memory mein toh crash ho jayega. Lekin virtually: `4 % 4 = 0` (Wapas `'a'` mil gaya!).
- Index `5 % 4 = 1` (Wapas `'b'`).

> **The $O(1)$ Optimization Strategy 💡:**
> Hum `a` ke upar ek pointer `i` chalayenge jo kabhi rukega nahi, balki `% n` hokar circular ghumta rahega. Aur `b` ke upar ek pointer `j` chalayenge.
> Hum `a` ke har ek index ko `b` ke starting point se match karne ki koshish karenge. Agar match chalu hota hai, toh hum circular fashion mein check karte chale jayenge jab tak `b` poora khatam nahi ho jata.

🛠️ Algorithm Steps

1. Maan lo `a` ki length N hai aur `b` ki length M hai.
2. Hum `a` ke har ek index `i` (0 se lekar $N-1$) ko starting point maanenge.
3. Har starting point `i` ke liye, hum ek andar loop chalayenge `j` jo `0` se lekar $M-1$ tak jayega (`b` ko check karne ke liye).
4. Hum characters ko virtually compare karenge: `a[(i + j) % n] == b[j]`.
5. Agar kisi bhi starting point `i` ke liye poori `b` string match ho jati hai (yaani andar waala loop bina break hue poora chal gaya):

- Toh total kitne characters cover hue? Shuruat hui `i` se, aur khatam hua `i + m - 1` par. Total characters = `i + m`.
- `a` ko kitni baar repeat karna padega? $\lceil (i+m)/n \rceil$ yaani `ceil((double)(i + m) / n)`.

6. Agar outer loop khatam ho jaye aur koi match na mile, toh `-1` return kar do.

📊 Microscopic Dry Run (Example: `a = "abcd"`, `b = "cdabcdab"`)

- $N = 4, M = 8$. Outer loop `i` chalega `0` se `3` tak.

When `i = 0` (Starting from 'a'):

- `j = 0`: `a[(0+0)%4]` → `a[0]` ('a') != `b[0]` ('c'). **Mismatch!** Inner loop break.

When `i = 1` (Starting from 'b'):

- `j = 0`: `a[(1+0)%4]` → `a[1]` ('b') != `b[0]` ('c'). **Mismatch!** Inner loop break.

When `i = 2` (Starting from 'c'):

- `j = 0`: `a[(2+0)%4]` → `a[2]` ('c') == `b[0]` ('c'). **Match!**
- `j = 1`: `a[(2+1)%4]` → `a[3]` ('d') == `b[1]` ('d'). **Match!**
- `j = 2`: `a[(2+2)%4]` → `a[0]` ('a') == `b[2]` ('a'). **Match!** *(Dekho virtual loop ghum gaya)*
- `j = 3`: `a[(2+3)%4]` → `a[1]` ('b') == `b[3]` ('b'). **Match!**
- `j = 4`: `a[(2+4)%4]` → `a[2]` ('c') == `b[4]` ('c'). **Match!**
- `j = 5`: `a[(2+5)%4]` → `a[3]` ('d') == `b[5]` ('d'). **Match!**
- `j = 6`: `a[(2+6)%4]` → `a[0]` ('a') == `b[6]` ('a'). **Match!**
- `j = 7`: `a[(2+7)%4]` → `a[1]` ('b') == `b[7]` ('b'). **Match!**

Inner loop poora khatam hua (`j == 8`). Perfect match mil gaya!

Result Calculation:

- Match starting point: `i = 2`.
- Total virtual characters consumed: `i + m` → `2 + 8 = 10`.
- Total repeats of `a`: `ceil(10.0 / 4.0)` = **`3`**.

💻 Production-Grade C++ Code ($O(1)$ Space)

```cpp
#include <iostream>
#include <string>
#include <cmath>
using namespace std;

int repeatedStringMatchO1Space(string a, string b) {
    int n = a.length();
    int m = b.length();

    // Outer loop: 'a' ke har ek character ko potential starting point
maano
    for (int i = 0; i < n; i++) {
        int j = 0;

        // Inner loop: Circular pointer check using modulo (%)
        while (j < m && a[(i + j) % n] == b[j]) {
            j++;
        }

        // Agar j poora m tak pahunch gaya, matlab b poora match ho gaya
hai
        if (j == m) {
            // Formula: Kitne virtual characters use hue divided by
original length n
            return ceil((double)(i + m) / n);
        }
    }

    return -1; // Agar saare starting points check karne par bhi na mile
}

int main() {
    string a = "abcd";
    string b = "cdabcdab";

    int ans = repeatedStringMatchO1Space(a, b);
    cout << "Minimum repeats required ($O(1)$ Space Solution): " << ans
```

```
<< endl;

    return 0;
}
```

⏱️ Complexity Analysis

- **Time Complexity:** $O(N \times M)$ **Worst Case (Practically Extremely Fast)**
- Outer loop N baar chalta hai aur inner loop worst case mein M tak ja sakta hai.
- Lekin dhyan do, isme humne koi lambi-lambi strings physically compute ya store nahi ki hain memory mein. Yeh characters ko directly read karta hai, jisse yeh cache-friendly ban jata hai aur memory allocation ka overhead bilkul zero ho jata hai.
- **Space Complexity:** $O(1)$ **Absolute Constant Space**
- Humne koi extra string concatenate nahi ki (`repeated_a = ""`), na hi koi KMP ka vector table (`lps`) banaya. Pure system mein hum sirf do pointer `i` aur `j` ghuma rahe hain, isiliye space absolute constant hai.

Bhai, KMP se space $O(N + M)$ ja raha tha, lekin is circular check se space pure $O(1)$ ho gaya. Interviewer jab space optimization mangega, toh yeh modulo (`%`) waala logic unka dil khush kar dega!